TECHNISCHE UNIVERSITÄT BERLIN

Practical FaaS Performance Variability Implications in the Serverless Image Optimization Scenario

Master's Thesis

Author
Piotr Witkowski
414360
witkowski@campus.tu-berlin.de

Advisor
Trever Schirmer

Examiners
Prof. Dr.-Ing. David Bermbach
Prof. Dr. habil. Odej Kao

Technische Universität Berlin, 2025
Fakultät Elektrotechnik und Informatik
Fachgebiet Scalable Software Systems

Practical FaaS Performance Variability Implications in the Serverless Image Optimization Scenario

Master's Thesis

Submitted by:
Piotr Witkowski
414360
witkowski@campus.tu-berlin.de

Technische Universität Berlin
Fakultät Elektrotechnik und Informatik
Fachgebiet Scalable Software Systems

2025

Hiermit versichere ich, dass ich die vorliegende Arbeit eigenständig ohne Hilfe Dritter und ausschließlich unter Verwendung der aufgeführten Quellen und Hilfsmittel angefertigt habe. Alle Stellen die den benutzten Quellen und Hilfsmitteln unverändert oder sinngemäß entnommen sind, habe ich als solche kenntlich gemacht.

Sofern generische KI-Tools verwendet wurden, habe ich Produktnamen, Hersteller, die jeweils verwendete Softwareversion und die jeweiligen Einsatzzwecke (z. B. sprachliche Überprüfung und Verbesserung der Texte, systematische Recherche) benannt. Ich verantworte die Auswahl, die Übernahme und sämtliche Ergebnisse des von mir verwendeten KI-generierten Outputs vollumfänglich selbst.

Die Satzung zur Sicherung guter wissenschaftlicher Praxis an der TU Berlin vom 8. März 2017* habe ich zur Kenntnis genommen.

Ich erkläre weiterhin, dass ich die Arbeit in gleicher oder ähnlicher Form noch keiner anderen Prüfungsbehörde vorgelegt habe.

(Unterschrift) Piotr Witkowski, Berlin, 7. März 2025

*https://www.tu.berlin/arbeiten/wichtige-dokumente/richtlinien-leitlinien

Abstract

Kurzfassung

Contents

1 Introduction **7**

2 Background **8**
2.1 Imaging and Photography 8
2.1.1 Digital Imaging 9
2.1.2 Data and Image Compression 10
2.1.3 Overview of Image Formats 12
2.2 Cloud Computing 14
2.2.1 Serverless Computing 16
2.2.2 Cloud Service Benchmarking 18
2.2.3 Performance Variability of Serverless Workloads 18
2.3 Image Optimization for the Web 19
2.3.1 Impact of Images and Bandwidth Usage on Browsing Experience . . 21
2.3.2 Deployment Models of Image Optimization Solutions 22

3 System Design **24**
3.1 Benchmark Design 24
3.1.1 Providers, Images, and Optimization Parameters Selection 25
3.1.2 Regions, Runtime, and Image Optimization Library Selection 27
3.2 Implementation 29
3.2.1 Results Collection and Data Analysis 30
3.2.2 Serverless Benchmark Deployment 31

4 Results **33**
4.1 Missed Executions 33
4.2 Cold Starts 34
4.3 Environment Lifetime and Environment Reuse Metrics 34
4.4 Measuring Image Processing Duration 36
4.5 Daily Temporal Variability 37
4.6 Weekly Temporal Variability 39
4.7 Environment-related Metrics 41
4.8 Handler Count-Related Duration Variability 42

5 Discussion **43**
5.1 Performance variability analysis 43
5.2 User vs operator perspective 43
5.3 Cloud computing as a utility 43

6 Related Work **44**
6.1 Serverless performance research 44
6.2 Image processing research 44

7 Conclusion & Future Work **45**

1 Introduction

The digital revolution has fundamentally transformed the way we create, consume, and interact with visual information. Images are no longer only physical items around us, they are inherent components of our online experience, shaping perceptions and influencing decisions. As the Internet continues to evolve, so too grows the demand for rich and engaging visual content. This demand, however, comes at a cost. Images, while visually appealing, are significantly impacting website performance, not only by generating the highest number of requests, but also by consuming the most bytes during page load of an average site. Existing studies have shown a strong correlation between page load times and user engagement, conversion rates, and even search engine rankings. Minimizing image size is therefore fundamental for efficient bandwidth use and plays a crucial role in optimizing website performance.

Image optimization practices for the web have evolved over time, following developments of the state-of-the-art image formats and codecs. These solutions perform operations such as image resizing and format conversion, and may allow for storing optimized versions of original files for future use. While the optimization itself is a relatively specialized task, its deployment models follow familiar and established patterns in the general software industry: we can find them in Software as a Service offerings, integrated into web hosting and content delivery platforms, but also as open-source solutions for self-hosting.

This last option, leveraging open-source image processing libraries, presents a compelling opportunity for our research. Unlike proprietary solutions, open-source projects offer the flexibility to be deployed across a variety of environments, granting builders granular control over the optimization process. This allows not only to measure influence of different libraries and optimization parameters, but also - and crucially for this thesis - enables realistic benchmarking of the underlying platforms on which these operations are performed. By employing these workloads, our research aims to accurately measure cloud service performance under the realistic conditions of real-world image optimization scenarios.

Given the event-driven nature of the optimization workload in response to requests for images, serverless cloud compute emerges as a well-suited deployment option for these systems. However, the varying demands for image processing, including different image sizes and diverse range of image formats present on the web, raise questions about the suitability and performance consistency of these platforms for the use-case. Furthermore, the emergence of function as a service (FaaS) platforms introduces a new dimension to cloud-based, self-hosted image optimization. FaaS offers a compelling execution environment for image processing tasks thanks to its on-demand scalability and event-driven nature. However, the performance variability inherent in FaaS platforms, attributed to diverse factors, may impact efficiency of the process and applicability of the environment for the use case.

This thesis investigates the influence of FaaS performance variability on image optimization workloads. Representative metrics, reflecting realistic web usage, will be defined. Optimization workflows will be designed and implemented for selected serverless environments. A workload generator will simulate requests for optimized images. Performance metrics will be collected across all parameters, with statistical methods applied for result comparison and visualization. We will provide recommendations within the benchmark's scope, to answer the primary research question in the context of the scenario.

2 Background

This chapter provides readers with foundations for understanding the challenges and opportunities of serverless image processing. We begin by revisiting fundamental principles of imaging and photography. Subsequently, we describe core digital imaging concepts, emphasizing aspects relevant to web performance. We then examine the characteristics of serverless cloud computing, focusing on factors contributing to performance variability. Finally, we explore image optimization for the web, reviewing established techniques and the landscape of cloud-based solutions.

2.1 Imaging and Photography

Imaging is a broad term describing any process that helps us in visualizing and understanding physical objects, abstract concepts, or even sets of data. While this may be a purely philosophical activity, for the purpose of this thesis, we will define imaging as creating a visual representation - an image - of physical objects and scenes. Humans are able to perceive and interpret these images thanks to our visual system, that has evolved over millions of years [52]. Developed from the simple light-sensitive cells of early organisms to the sophisticated eyes of modern humans, this system is providing us with the ability to see color, depth, or movement [54].

From the earliest cave paintings [88] to modern photography and film, humans have been exploring ways to capture and recreate the visual world. Over centuries, scholars and artists showed great interest [42] in the process of creating visual representations to understand our own mind and senses, as well as to be able to depict the world around us more accurately. The natural phenomenon of light passing through a small aperture, called camera obscura, helped us understand fundamental principles of optics and image formation, and according to some [39], greatly contributed to realism of European art during Middle Ages. We've been refining tools and techniques to enhance our vision, and improve everyday life. Devices like telescopes and microscopes allowed us to observe objects at different scales, unveiling worlds that would otherwise remain invisible to the naked eye. Eyeglasses and contact lenses correct refractive errors, demonstrating that our visual experiences can not only be externally altered, but also subjectively improved, for millions of people.

Photography emerged in the 19th century as a groundbreaking invention, offering a novel approach to image creation. Unlike paintings or drawings, which rely on an artist's interpretation, photography directly captures and permanently records the physical world using light-sensitive materials. Early photographic methods required long exposure times, often making it possible to photograph only still objects and posed portraits [20]. The daguerreotype, which used a silver-coated copper plate, was one of the first widely adopted photographic processes. Later, more versatile film technologies significantly reduced exposure times and made photography more accessible to the wider public. This ability to easily and accurately record the physical world had significant consequences. Newspapers, now able to include actual photographs, found a powerful new way to share news and social commentary through photojournalism [32]. On a personal level, photography allowed people to preserve their own histories through portraits and the documentation of special occasions.

Since the beginning of the 21st century, digital photography has become the dominant form [97] of capturing and sharing images. Initially limited to standalone devices, digital cameras are now integrated into smartphones and personal computers. Electronic sensors

capture and convert light into digital data, facilitating easy manipulation and image sharing, including over the Internet. This digital nature has led to the popularity of standardized formats, ensuring compatibility across diverse platforms.

However, the ease of image creation and sharing, coupled with the vast amount of image data generated daily, may present unexpected challenges for storage and transmission. In 2024, an estimated 6 billion photos are taken daily [13], with 14 billion images shared on social media platforms [71]. Especially in the context of thesis, the resulting volume of image data necessitates efficient delivery. Fortunately, optimization methods exist, relying on core concepts of image representation, encoding, and compression, to ensure efficient delivery across networks. We will explore these in the subsequent sections.

2.1.1 Digital Imaging

Digital imaging, like traditional photography, relies on the fundamental principle of capturing light to create a visual representation. Most image acquisition methods, whether they involve cell phone cameras, DSLR (digital single-lens reflex) and mirrorless cameras, or even scanners, are variations of the classical optical camera [94]. Concepts such as the relationship between aperture and shutter speed to achieve proper exposure are universal, regardless of whether the capture method is analog or digital.

However, digital imaging distinguishes itself by representing images as discrete, two-dimensional arrays of numerical data. This involves transforming light into a grid of distinct pixels. Each pixel may contain one or more channels — components that record different aspects of light, often corresponding to how our eyes perceive color. If only one channel is present, the image is effectively grayscale, as was the case with the first digital camera invented in 1975 by Steven Sasson [63].

To create a digital image, the camera must first capture the light that's focused onto its sensor. The position of the camera and its lens determine what portion of the scene is captured by each pixel on the sensor — this is known as *spatial sampling*. The camera also controls how long it gathers light at each pixel, called *temporal sampling*, and is determined by the exposure time, or using photographic terminology, *shutter speed*. Thanks to the photovoltaic effect, the amount of the light captured can be converted into digital value in a process called *quantization* [15]. The number of distinct values that can be represented for each pixel depends on the bit depth of the image.

Digital cameras primarily use two image sensor technologies: CCD (Charge-Coupled Device) [85] and CMOS (Complementary Metal-Oxide-Semiconductor) [30]. Early digital cameras utilized CCD sensors, which offer high sensitivity and low noise due to their charge transfer method, where all charge is shifted across the chip to a single output node. However, this sequential readout makes them slower for continuous shooting or video capture. CCDs also tend to consume more power and are more prone to blooming, where overexposed pixels overflow. In contrast, CMOS sensors have seen rapid development since the 1990s. While initially lagging in sensitivity and noise performance comparing to CCDs [56], CMOS sensors have greatly improved through advancements like on-chip noise reduction. Their lower cost, lower power consumption, and faster readout speeds have led to CMOS sensors becoming more popular than CCDs and being found in a wide range of devices, from smartphones and webcams to DSLRs, mirrorless cameras, and even scientific imaging equipment [41].

To accurately capture color, most digital cameras use a *color filter array* (CFA) placed over the sensor. The most common type is the Bayer filter, named after its inventor, Bryce Bayer, working at the time at Eastman Kodak. This filter arranges red, green, and blue filters in a specific pattern over the pixels. Because the human eye is more sensitive to green light, or to be specific, because *green signal contributes more to the luminance signal than the red or blue signals* [47], in a typical Bayer filter we can find twice as many green filters as red or blue. Cameras capture a full-color image by interpolating the missing color information for each pixel, in a process known as *demosaicing* [4].

Following the Bayer filter example, it is common for color images to have three channels, such as red, green, and blue (RGB), each with its own bit depth. If each channel in an RGB image has an 8-bit resolution, it allows for 256 different values per channel, resulting in over 16 million possible color combinations for each pixel. The resolution of an image refers to the total number of pixels it contains and is often measured in megapixels (millions of pixels). Higher resolution images have more pixels, what allows for larger prints and more detailed displays. The image resolution, along with the bit depth, directly affects the size of the uncompressed image file. However, because images often contain areas of similar color, compression techniques, discussed in the next section, are frequently employed to reduce file sizes, sometimes even during the image capture process within the camera devices.

2.1.2 Data and Image Compression

Digital data, whether representing text, audio, or images, is fundamentally a collection of bits. Data compression techniques aim to represent the same information using fewer bits [40], which is important for addressing physical constraints of digital systems, such as bandwidth, storage density, and power consumption. In 1948, C. Shannon introduced [78] the concept of entropy as a measure of information content. Entropy defines the fundamental limit for lossless data compression algorithms, providing a theoretical target for minimizing the number of bits needed to represent information. To effectively leverage data compression techniques, it's important to differentiate between lossless and lossy compression, recognizing the trade-offs between preserving data integrity and achieving higher compression ratios.

Lossless compression methods enable reversible transformations of data into more compact forms, while preserving all original information. While this thesis primarily focuses on image processing, we would like to note that lossless techniques have much broader applications and can be applied to virtually any form of digital data, including text and binary files. This is crucial for efficient data transmission of any kind, where lossless compression reduces bandwidth requirements, but allows recipients to access the original information in its unaltered [75] form.

Run-length encoding is a simple technique that exploits repetitions in data. For example, a sequence of the same 20 consecutive characters *a* can be efficiently stored as *(20, a)* instead of storing each individual character. This works especially well if the set of unique symbols in the data is small, and the data contains long *runs* of the same symbol.

Huffman coding [45] takes a statistical approach by constructing a binary tree *(Huffman tree)* based on symbol probabilities. Leafs representing higher probability symbols are positioned closer to the root, with shorter code lengths. This results in a variable-length code, where the length of a symbol's bit sequence is inversely proportional to its frequency in the data. By assigning shorter codes to common symbols, Huffman coding optimizes code length, reducing the overall number of bits needed.

Arithmetic coding [67, 72] takes yet another approach, encoding the entire message as a single fraction between the values of *0* and *1*. The exact value is determined by repeatedly subdividing the interval based on the probabilities of the symbols. Each symbol narrows the interval, and the final fraction precisely represents the entire sequence, often allowing it to be represented with fewer bits than the original message. However, this method requires the probability distribution of the symbols being encoded to be shared between the encoder and the decoder.

In addition to the methods described above that can help us understand the basics of compression, practical implementations of lossless compression often rely on dictionary-based techniques. The *Lempel-Ziv* algorithms, including LZ77 [99], LZ78 [100], and the *Lempel-Ziv-Welch* (LZW) [93] algorithm, are important examples of this approach. LZ77 utilizes a sliding window to locate matches in preceding data, while LZ78 constructs a dictionary of previously encountered phrases. Combining LZ77 with Huffman coding, *Deflate* [21] often achieves higher compression ratios than LZ77 alone.

Asymmetric Numeral Systems [22, 23] (ANS) is a relatively new approach to entropy coding that combines the speed of Huffman coding with the compression rate of arithmetic coding. Unlike arithmetic coding, which uses two fractional numbers to represent an interval, ANS encodes data directly as a single natural number. This simpler representation, using just one integer, reduces computational complexity and allows for higher compression and decompression throughput. ANS is expected [24] to gain more popularity across various domains, including web delivery and image compression [44], as research [16] and implementation efforts continue.

Lossy compression methods, unlike lossless techniques, offer significantly higher compression ratios [75] by selectively discarding information. This trade-off is often acceptable for media files like images and audio, where human perception of minor data alterations may be limited. At the same time, even smallest data alterations are not acceptable for other types of binary data, such as executable code, digital signatures or encryption keys.

Transform coding is a common technique used in lossy compression that serves two purposes. By compacting the original data samples into a series of transform coefficients with smaller and smaller variances, we are eventually able to remove the smallest coefficients with negligible reconstruction errors. Additionally, decorrelated coefficients can be quantized and entropy coded without losing compression performance. By applying a transform, such as the *Discrete Cosine Transform (DCT)* [2], we can separate the information into components that vary in importance with respect to human perception. In lossy image compression, DCT selectively removes high-frequency components [92] which correspond to image details that are the least perceptible to human eyes.

Subsampling exploits the characteristics of human perception by reducing the spatial resolution of specific components within the data. In image compression, this often involves relative downsampling the chrominance components *(chroma)*, as the human eye is less sensitive to color detail compared to luminance *(luma)* detail. The reduction in chrominance resolution contributes to a smaller file size without significantly impacting the overall perceived image quality.

Evaluating the effectiveness of lossy compression involves measuring the perceived difference between original and compressed images. Selected evaluation metrics include Peak Signal-to-Noise Ratio (PSNR), Structural Similarity Index Measure (SSIM), and Butteraugli [43]. While metrics like PSNR and SSIM provide objective measurement of the differences

between pictures, the ultimate measure of lossy compression effectiveness will eventually always depend on the subjective perception of its observers.

Another factor to consider with lossy compression is generation loss, which refers to the accumulated degradation of quality that occurs when a compressed file is repeatedly edited and recompressed. Each recompression cycle may introduce additional information loss, potentially leading to noticeable artifacts and a significant reduction in overall subjective quality of the image. In the next section, we will explore different image formats that utilize the lossy and lossless compression techniques described above.

2.1.3 Overview of Image Formats

Image formats are standardized ways of organizing digital image data and metadata within files. They ensure compatibility and allow different software, devices, and operating systems to work with the same images. Two fundamental categories of image formats are *raster* and *vector*.

Raster images [28] are fundamentally composed of a grid of individual pixels, each representing a specific color. This structure directly corresponds to the way digital cameras capture images and is well-suited for representing photographs and other visuals with continuous tones. The term raster originates from the scanning pattern *(raster scan)* of CRT monitors, where the electron beam illuminates the screen pixel by pixel, line by line. In addition to color data, some raster formats incorporate an alpha channel [69], which controls each pixel's transparency. Image quality in raster formats is fundamentally connected to the resolution, measured by the total number of pixels. When scaling a raster image beyond its original size, preserving a fixed number of pixels can lead to a visible pixelation - a situation when individual pixels become visible to a naked eye. The alternative approach of introducing new pixels — *interpolation* [36] — also has drawbacks: traditional *generating a raster image with a higher resolution than its source* using pixel interpolation methods may cause a visible loss of sharpness [66], also known as *edge blurring*.

Vector graphics offer a fundamentally different approach, representing visuals using geometrical primitives like lines, curves, circles, ellipses, or polylines and polygons [60]. These primitives are defined by their mathematical properties, not as individual pixels. For instance, a circle may be defined by its center point coordinates and radius, while a curve might be represented using its end point and control points. This allows vector graphics to be inherently scalable, as the same geometrical definitions are used to render the graphic at any size.

While the following sections of our thesis will focus on raster image optimizations, a brief comparison of image optimization techniques for both vector and raster images is beneficial for the comprehensive understanding of the topic. Pixel-based methods, such as reducing image resolution, the bit depth, or applying lossy compression can not be directly translated to their vector counterparts, as vector graphics do not rely on a pixel grid [53]. Instead, byte size of vector graphics may be optimized through methods unique to their geometrical nature. While lossless compression methods still apply, further optimizations are typically achieved by reducing the precision of values defining geometrical primitives, simplifying complex shapes by using fewer defining points, and, in formats supporting reusable styles, minimizing the number of distinct styles used [57].

While based on common data representation principles, specific raster formats use different strategies for storing and compressing pixel data. The existence of multiple formats is

driven by technological advancements in the field of image processing, evolving consumer needs, as well as industry competition [61]. While new algorithms and standards allow for smaller file sizes and additional features, widespread adoption can only happen if the benefits of new image formats outweight compatibility issues and justifies additional investment (monetary and engineering) needed to enable and integrate the new technology into existing systems and workflows.

BMP (Bitmap) is one of the simplest raster formats. It can use lossless compression through run-length encoding (RLE), or store images uncompressed. While BMP supports various color depths, its compression is generally inefficient compared to more modern formats, leading to large file sizes. Bitmap's lack of advanced features and inefficient compression limits its use to scenarios where file size is not a concern, but compatibility with older systems is important.

GIF (Graphics Interchange Format), introduced in 1987, was one of the first widely-adopted image formats on the Web, with the support on the support for animation and lossless compression based on the LZW algorithm. However, GIF is limited to an 8-bit palette, meaning it can only display 256 colors. This restriction makes it unsuitable for photographs but still viable for graphics with limited color ranges, such as logos or simple icons. GIF also supports transparency, allowing for irregularly shaped, non-rectangular images. While GIF has been largely superseded by PNG for static images, its animation capabilities continue to make it relevant for short, looping animations, especially in online contexts.

JPEG (Joint Photographic Experts Group), known for its wide acceptance and compatibility with software and web browsers, is a lossy compression algorithm based on the Discrete Cosine Transform (DCT) [89]. The actual image data in JPEG files is often encapsulated in a JFIF (JPEG File Interchange Format) container, which provides a standardized way to store JPEG-compressed images. JPEG supports a limited 8-bit RGB color space, and its lossy nature can introduce visual artifacts, such as blockiness, ringing, and blurring, especially at higher compression ratios [58]. Progressive JPEGs offer an alternative encoding method, where the image is displayed in multiple passes with increasing levels of detail, allowing for a quicker preview of the image even with limited bandwidth.

PNG (Portable Network Graphics) was developed as a patent-free alternative to GIF, offering improved lossless compression and additional features [73]. Comparing to GIF, it is using the more advanced DEFLATE algorithm, and achieves better compression than its predecessor without sacrificing image quality. Its wider range of color depths, including alpha channel support, allows for smooth transparency gradients. These features have made PNG the preferred choice for web graphics requiring high quality and transparency, such as logos, icons, and interface elements.

JPEG 2000 was designed as a successor to JPEG, offering improved compression efficiency and a wider range of features [79]. Instead of DCT, JPEG 2000 utilizes wavelet transforms, which provide better image quality at higher compression ratios. It supports both lossy and lossless compression, as well as more features, such as region of interest encoding, allowing selected areas of the image to be encoded with higher quality. While JPEG 2000 offers technical advantages, its adoption has been limited due to factors like licensing costs and the widespread use of the original JPEG format. Its applications are mostly limited to fields such as medical imaging and digital archiving.

WebP is a relatively new (2010) format developed by Google, specifically for web use [98]. It supports both lossy and lossless compression, providing smaller file sizes comparing to JPEG and PNG. Lossy compression is implemented using VP8 codec [6, 7], and lossless compression leverages LZ77 as well as Huffman coding. WebP supports both transparency and animation. It is widely supported on the Web[1], and in hardware [8].

AVIF (AV1 Image File Format) is a modern image format [9], introduced in 2019, derived from the AV1 [37] video codec. It was designed to offer superior compression compared to existing formats like JPEG, PNG, and even WebP. AVIF supports both lossy and lossless compression and a wide set of features, including high dynamic range (HDR) and wide color gamut (WCG) images, as well as transparency. AVIF generally offers significant file size reductions compared to JPEG while maintaining comparable visual quality. Its support for advanced features like HDR and WCG makes it suitable for modern, high-fidelity display technologies. It is also royalty-free, which allows for faster adoption[2], across web browsers and operating systems. Similar to JPEG using JFIF as a container, AVIF uses HEIF (High Efficiency Image File Format) to store images compressed with the AV1 codec.

JPEG XL is another modern image format designed as a successor to JPEG [3]. It offers both lossy and lossless compression, outperforming JPEG in terms of compression efficiency. One of JPEG XL's key strengths is its ability to losslessly recompress existing JPEG files, reducing their size without any further quality loss. JPEG XL also supports animation, an arbitrary number of channels, as well as parallel, progressive, and partial decoding. In contrast to WebP and AVIF, which use video codecs, JPEG XL is designed specifically for still images. The DNG 1.7 specification [1] allows using JPEG XL as the payload codec to store the raw camera data. According to the estimates [81], using a JPEG XL payload reduces the file size of a lossless 48 megapixel photo from 75 MB down to 46 MB, which translates to almost 40% file size reduction without any loss.

2.2 Cloud Computing

Various organizations and standardization bodies have formulated alternative definitions of cloud computing based on their specific priorities and perspectives. ISO/IEC 22123-1 [83] defines cloud computing as a "paradigm for enabling network access to a scalable and elastic pool of shareable physical or virtual resources with self-service provisioning and administration on-demand". This definition, with the emphasis on essential characteristics like on-demand availability and self-service, shares similarities with the one provided by the National Institute of Standards and Technology in *The NIST Definition of Cloud Computing* [59]. However, NIST also provides a more focused categorization of cloud services, identifying three service models (Infrastructure as a Service - *IaaS*, Platform as a Service - *PaaS*, and Software as a Service - *SaaS*) and four deployment models (private, community, public, and hybrid clouds) instead of 7 service categories and 8 deployment models in the ISO standard. In contrast to the frameworks offered by ISO/IEC and NIST, commercial organizations like Amazon Web Services (AWS) tend to use more concise and business-oriented definitions, such as "on-demand delivery of IT resources over the Internet with pay-as-you-go pricing" [5]. Despite these variations, all definitions focus on the core value proposition of cloud computing: providing flexible, accessible, and easy-to-manage computing services.

[1] WebP is supported across all major browsers and over 95% of Web users, https://caniuse.com/webp

[2] AVIF is supported across all major browsers and almost 95% of Web users, https://caniuse.com/avif

In a manner similar to services such as water and electricity supply, cloud computing can be seen as a utility and an on-demand enabler for innovation and agility without an upfront investment [34] in one's own IT infrastructure. With access to public cloud infrastructures, organizations do not have to build and maintain their own data centers, much like how most individuals and businesses no longer need to build their own power plants. Instead, they can focus on their unique strengths and leverage the scale and efficiency of shared resources, essentially offloading the *undifferentiated heavy-lifting* [51]. Furthermore, many cloud providers offer free tier access to a range of their services, allowing IT practitioners and business decision makers to experiment and gain familiarity with cloud environments before committing to substantial financial investments.

Cloud service models described earlier form a spectrum. Starting from the least *managed* types of services in the *IaaS* model, bare-metal servers and virtual machines offer maximum control, but require the highest degree of direct infrastructure management. In the *PaaS* model, developers and software vendors[3] trade some of this control for simplified operations [90]. It is still required to provide source code for applications, but the platform provider typically manages the operating system and offers additional capabilities with help of the platform-specific APIs and SDKs. In the *SaaS* model, a pre-built cloud service is solving a specific business problem for its users[4], with the least amount of flexibility comparing to previous models. SaaS services work well for repeatable workloads, such as project management software (Asana, Trello), communication and collaboration tools (Slack, Google Workspace), or e-commerce (Shopify) that can be applied with minimal modifications for a wide range of customers [50].

No two cloud platforms are the same. Leveraging differentiating, proprietary features can accelerate development of cloud-based services, but at the same requires careful consideration before adoption. These features, often exclusive to specific providers, can lead to a situation where users inadvertently depend on a particular technology, known as *vendor lock-in* or *feature lock-in* [68]. This dependency can pose a challenge when migrating to a different cloud vendor or pursuing a multi-cloud deployment strategy. Moreover, the perceived value and associated risks of such dependencies can change over time, influenced by evolving business needs and technological landscape.

Shared Responsibility Model is a framework adopted independently by multiple public cloud providers. In its basic form, it refers to the security *of the cloud* and the security *in the cloud*. In this model, both cloud provider and cloud user share responsibility of different security aspects of the cloud-based workloads.

- *Cloud provider* is responsible for the physical security of the data center facilities, providing uninterrupted power and cooling supply, and maintaining integrity of the infrastructure. Cloud provider is also responsible for providing customers with building blocks to implement architectural best practices, such as resiliency to natural disasters with multiple regions and availability zones.
- *Developers and software vendors* are responsible, among other things, for preparing disaster recovery strategy including platform support, following the principle of last privilege, timely updates of their application code, or applying application-layer encryption.

[3] By developers and software vendors, we mean personas setting up and maintaining cloud deployments.
[4] By users, we mean end-users of the services and applications deployed on the cloud.

In addition to the above, we would like to note that the exact split in the shared responsibility model depends greatly on the used service delivery model. While things like power or cooling for IT resources are hidden from end users independent of the used model, many other responsibilities, similar to the lock-in discussion above, depend on whether users opt into abstractions provided by the cloud provider. Typically, the more *managed* a service or model is, the more responsibility is taken by the cloud service provider. Providers abstracting typical IT use cases, such as OS supervision, load balancing, and availability of the managed services according to the SLAs of individual services. This leads to less freedom in configurability, but instead, users can shift their development and operational effort towards developing high-value, differentiating capabilities in their domain of expertise. For example, when comparing bare-metal instances with virtual machines, users will be required to set up and maintain a hypervisor only in the first situation. On top of security, similar shared responsibility models can be applied to domains like compliance or even sustainability.

The following sections and the rest of our thesis will focus on serverless computing, a paradigm where, contrary to its name, servers still exist. Serverless operations are abstracted even further from the developers, therefore, as before, it is important to make an informed decision whether to opt into platforms' vendor-specific features. Existing literature, such as [48], explores the potential pitfalls and fallacies associated with serverless computing, providing a valuable context for understanding the broader implications of these decisions. Finally, adoption of cloud computing, like any technology, while potentially beneficial, must be evaluated with the user in mind. As Tim O'Reilly put it, "lock-in comes because others depend on the benefit from your services, not because you are completely in control" [64] - it is the users who eventually benefit from, or are constrained by, all architectural choices.

2.2.1 Serverless Computing

Serverless is an approach where cloud providers aim to simplify cloud operations even further. While sharing many similarities with the PaaS model, PaaS services typically operate on the application or microservice level. Serverless approach is instead concerned about individual capabilities or business functions within an application. In this model, software vendors have very little control over actual hardware, for example size and type of the(virtual) machine, on which their business logic is executed. Capacity provisioning, capacity planning, architecting for redundancy and high availability, as well as detailed monitoring features are built into serverless platforms, and managed by the platform provider.

Two most popular service deployment models applicable to the serverless approach are:

- **Backend as a Service (BaaS)** model provides developers and software vendors with options to adopt cloud-based services directly by clients, such as websites and mobile applications. Examples of BaaS frameworks include Google's Firebase or AWS Amplify[5], and typical use cases include authentication, persistent storage coupled with user's identity, or notifications management. Implementation-wise, BaaS is similar to integrating with any other SDK, with the major difference of focusing on rather *core* capabilities of an application transparently for the users - often without making

[5]Similar to Google's Firebase, AWS Amplify lets users host static pages and predefined application backends. In contrast to Firebase, Amplify offers purely client-side wrappers for other managed AWS services, such as Amazon Cognito, Amazon API Gateway, or Amazon S3, to simplify client-side integrations with these services.

them aware about its reliance on a particular BaaS product, as if the application custom-made backend was used.

- **Function as a Service (FaaS)** enables developers and software vendors to run code without the need to manage the underlying infrastructure or runtime. Examples of FaaS services include AWS Lambda and Google Cloud Functions. Functions can be synchronous or asynchronous. Responding to an HTTP request is an example of the former, and event-based invocation is an example of the latter. Asynchronous functions are typically executed for their side effects, such as storing the result of a computation in an external data store. While functions can persist state between invocations, service providers may terminate execution environments at any time they are not actively handling an invocation. Therefore, functions should be designed to operate independently of any existing state within the execution environment. Sometimes, simply providing the function code is sufficient, with the FaaS platform managing dependencies and the execution environment. If the provider supports custom container images as the function source, users are not limited to any specific programming language or runtime when authoring functions, but they will need to build, deploy, and maintain these container images themselves.

Thanks to granular monitoring mentioned earlier, serverless services offer fine-grained, usage-based pricing models. These operate on dimensions such as individual requests, CPU time with high resolution (as low as 1 millisecond, which is not the case in IaaS or PaaS models), amount of specific API operations, or metrics such as MAU (monthly active users). On-demand capacity of serverless backends scales automatically, and in real-time, in response to traffic. However, all this flexibility, especially in the FaaS model, comes at a cost. *Limited lifetime* [38] of serverless functions prescribes that their internal state may not persist between invocations. True *on-demand scaling to zero* only works, if we accept the trade-off of potential cold starts. With provisioned capacity we can avoid cold starts, but scaling to zero will not be possible anymore. Similarly, a choice must be made to the question of using a built-in runtime versus a custom container image. All of such architectural choices have to balance the complexity of the deployment, cost, as well as performance [18] of the resulting service.

In FaaS deployments, minification and tree-shaking can be used to reduce the size of the deployment package, and therefore the cold start time [70]. Native libraries can be used within serverless functions, even without a custom container, but in such cases, it's crucial to ensure that the CPU architecture and operating system runtime of the serverless platform is compatible with the library code. By leveraging such libraries, developers can avoid typical performance penalty associated with scripting languages [65], while enjoying convenience of high-level language to handle an event. To sum up, serverless computing offers a compelling approach to cloud operations, shifting the focus from infrastructure management to individual functions or capabilities. While BaaS simplifies client-side integrations with core application features, FaaS allows code execution without runtime management.

In the following sections, when speaking about serverless, we will focus exclusively on the Function as a Service model, as it is more relevant to the self-hosted image optimization use case we are exploring in this thesis.

2.2.2 Cloud Service Benchmarking

Cloud service benchmarking is the process of systematically evaluating and comparing different qualities of cloud-based services. Quality can be defined as a measure of any non-functional property of a service. In other words, qualities tells us not *if*, but *how good* a system fulfills its purpose. Qualities may describe radically different aspects of a system, for example, measure the error rate of a given operation or degree of compliance with a particular standard or regulation. While importance of a quality depends on a specific system and application, qualities in general express the difference from an ideal, sometimes purely abstract reference point. Continuing with the previous examples, an ideal system or service of the highest quality would yield an error rate of 0 (no errors, or undesired results), and demonstrate complete alignment with, or no exception from, a certain specification.

While *monitoring* is a continuous, low-impact observation of the production environment, used to detect undesirable system states [10], *benchmarking* answers specific questions about the system under test (SUT). It is often a result of an offline analysis, and based on the recording of arbitrary metrics. In benchmarking, a separate, non-production environment is used, as certain operations, for example stress-testing or fault injection, may be too risky and disruptive in a production setting or result in undesired consequences, such as a service outage or revenue loss.

Benchmarking allows for the evaluation of different configurations within the same environment, such as testing various memory allocations of a serverless application. It also enables comparing different systems while maintaining consistent configuration parameters, for example, measuring performance across multiple cloud providers or regions. This comparative nature helps identify optimal settings for specific use cases and understand the trade-offs between different implementation choices. By keeping some variables constant while varying others, benchmarking provides insights into how different factors influence system behavior.

Because cloud services are consumed over the Internet, designing benchmarks from the client perspective that closely reflect later use is essential to get meaningful results. In particular, even if additional information about the system is available (for example, source code of the application), literature [10] suggests treating the underlying cloud platform and application deployed on it as a *black box*. The reliance on the service's external interface / API contract, rather than depending on the actual implementation, allows us to compare different versions of the same service, and also align as closely as possible to the actual consumers' quality experience.

To facilitate benchmarking, a few components of a *benchmarking system* are needed. A *workload generator* is responsible for creating a realistic load on the system under test, as otherwise the system is idling. An *experiment controller* automates the execution of benchmark runs, managing configurations and collecting results. Finally, a *measurement database* stores the collected performance data, enabling analysis and comparison across different scenarios.

2.2.3 Performance Variability of Serverless Workloads

Performance variability is a key consideration in Function as a Service (FaaS) environments due to the high level of abstraction and limited control over the underlying infrastructure. Existing literature has explored and identified several sources of this performance variability within serverless platforms, including [55] studying cold and warm service performance, [87]

researching influence of storage accesses and bursty function invocations, and [76] exploring variations in functions' execution duration and the frequency of cold starts depending on the diurnal and weekly patterns. [91] investigates performance isolation efficiency of different serverless platforms. Additionally, all of these papers ([55, 76, 87, 91]) study the influence of performance-related settings, such as amount of assigned memory and compute, on the performance of Function as a Service platforms.

One of the primary contributors to performance fluctuation is the phenomenon of *cold starts*. When a function is invoked for the first time (or after a period of inactivity), the cloud provider must allocate resources, initialize the runtime, and load the function code. Subsequent "warm" invocations can reuse this environment, leading to significantly lower latency. However, FaaS providers offer no guarantees about environment lifetimes, meaning cold starts can occur unpredictably. Mitigation techniques like provisioned concurrency exist but introduce trade-offs in cost and resource utilization. Research [18] shows that the cold start latency depends on the deployment method (ZIP deployments being preferable for smaller Node.js, Python, and all Java functions, and container deployments for larger Node.js and Python functions), as well as the used programming language. Java functions, in particular, are known to exhibit longer cold start times due to the overhead of the Java Virtual Machine (JVM) [96]. While the JVM's size and startup contribute to this latency, mitigation strategies such as Ahead-of-Time (AOT) compilation (e.g., with GraalVM) and memory snapshots (e.g., AWS Lambda SnapStart) can reduce this delay.

Beyond cold starts, variability in resource allocation [33] also impacts performance. Cloud providers dynamically allocate resources (CPU, memory, network) to each function invocation. Multi-tenancy and hardware differences can lead to inconsistent execution times even for identical workloads. Network latency, especially when functions interact with other services and APIs, adds another layer of unpredictability due to factors like congestion and variable geographical location (for example, in different regions, or availability zones). Benchmarking from a client perspective is crucial to capture realistic performance.

Concurrency and queuing effects further contribute [77] to variability. Simultaneous invocations can lead to queuing particularly during peak loads, as the cloud provider's orchestration layer manages resources. These peak loads often exhibit periodic patterns, influenced by human activity and the scheduling of recurring tasks, which tend to cluster at common intervals (e.g., the start or end of hours, days, or weeks).

Due to the inherent abstraction of the FaaS model, ensuring consistent performance is a critical requirement for providers, as it is a prerequisite for predictable application behavior and end-user experience. However, FaaS platform users can also influence workload performance, for example, by strategically scheduling workloads to avoid known periods of high usage mentioned above.

2.3 Image Optimization for the Web

In its broadest sense, web delivery refers to the process of transmitting web content from a web server to a user's web browser using technologies including the addressing system (URIs), the network protocol (HTTP) and the markup language (HTML), known together as the World Wide Web [11]. These technologies have provided users of the Internet with *a consistent means to access a variety of media in a simplified fashion* [46]. Since its invention of *the Web* over 35 years ago, we have seen enormous effort and investment put into optimizing web content delivery, including both its hardware (distributed, purpose-

built networks for content delivery) and software part (protocols, including HTTPS [27], HTTP/2 [86], HTTP/3 [12], WebSockets [26], and various other specifications[6]).

Optimizing performance of the delivery can be achieved by following the general measures that answer the 6 common journalistic questions:

- **What**: Website delivery is based on data transfer. We can optimize website loading time by reducing amount of content sent to the user. After selecting specific content, its size can be further reduced with compression (see Section 2.1.2). Text and other arbitrary data can be compressed using lossless techniques mentioned before. Media content, such as images, video and audio, can be compressed even further by using lossy techniques. In the context of data, we are limited by its entropy.
- **How**: Quickly and reliably - using dedicated, high-bandwidth links with low packet loss and retransmission rates, and efficiently - using protocols that reduce signaling and minimize overheads on all layers of the networking stack.
- **When**: Typically, when a client makes a request, but also in advance, by preparing content that is likely to be requested in the future. This can be achieved by preloading content, or using server-sent. In this context, we are limited by the time of the first interaction between the client and the server.
- **Who**: Content can be specific to a given user, or applicable to a wider audience, enabling content reuse and request collapsing. Instead of generating unique response for each request, reusing previously generated content can reduce server load and improve response latency. In this context, we are limited by the degree of personalization of the content.
- **Where**: Content can be delivered from a variety of locations, including the authoritative origin server, or a proxy (cache) server, such as a reverse proxy, also known as content delivery network (CDN[7]), or a forward proxy, to optimize latency. In this context, we are limited by the physical distance between the client and the server and the speed of light if the content is not already present in the client's cache.
- **Why**: While this does not directly influence the performance of the delivery, understanding the purpose of content can help in request prioritization and response ordering for the specific use case and audience. This optimization is limited by the previously mentioned objectives of the service owners, as well as the users' expectations of the service.

A typical web page is composed of the HTML markup and additional resources, including fonts, stylesheets, scripts, and media. While there is no requirement to use any of these additional resources, in practice, according to the data from the HTTP Archive [80], a median page is sending approximately 70 additional requests. Among them, videos are the heaviest on a per-request basis, however, it is **images that contribute the most to the overall weight of an average homepage**, as 99.9% of landing pages load at least one image resource, and only 7% of pages load a video [49].

In context of images, a balance needs to be found between the performance, or at least perceived performance of the page, and the visual quality of the images that are displayed. Image performance is a measure of how quickly the images are rendered to provide value

[6] https://developer.mozilla.org/docs/web/http/resources_and_specifications
[7] https://developer.mozilla.org/docs/glossary/cdn

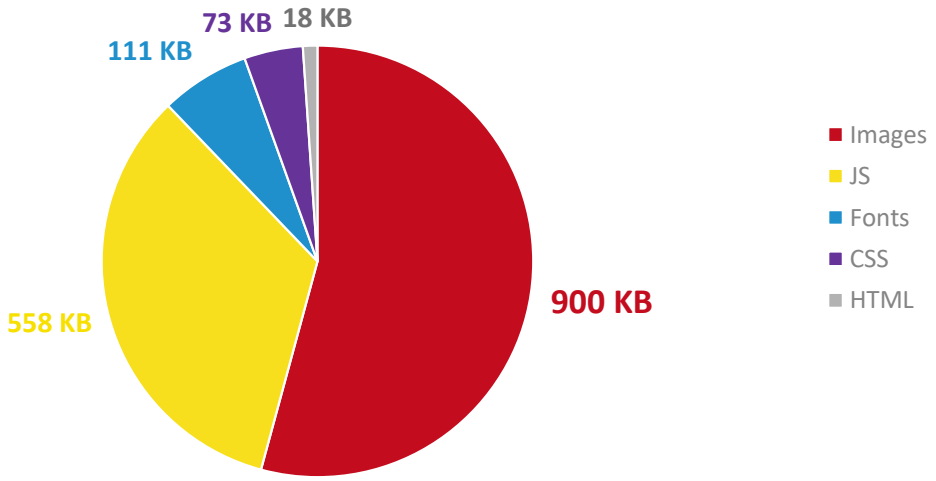


Figure 2.1: Median mobile homepage weight, by content type [80]

to the users. While subjective, image quality is often associated with the resolution, color depth, and compression artifacts present in the image, as described in section 2.1.3.

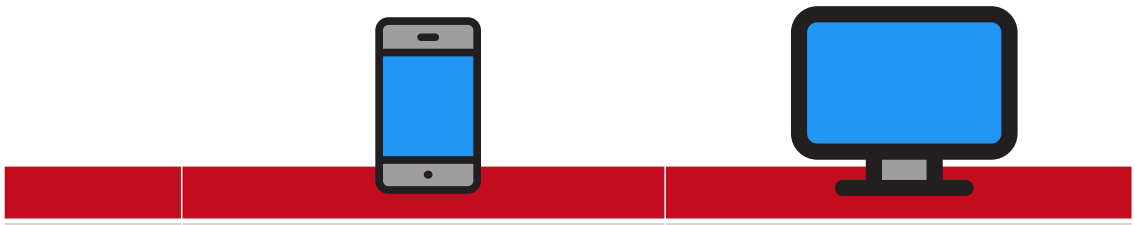
Critical rendering path[8] is a sequence of steps a browser takes to render a web page, starting from the initial page request until it can be rendered on a screen. Resources referenced by a page can be either *blocking* or *non-blocking* in regard to the critical rendering path. Traditionally, most resources (styles, scripts, fonts, and images) are blocking, meaning, the browser must wait for them to be downloaded and processed before rendering the page. This can lead to a situation where the user sees a blank page for a prolonged time. To mitigate this, developers can optimize the delivery of resources, for example by inlining critical resources, and loading non-critical resources asynchronously, in an on-demand manner. In case of images, this can be achieved by loading only images placed in the viewport, and *lazy loading*[9] the rest. An approach similar to lazy-loading, but on the opposite end of the spectrum, is *preloading* content that is most likely to be needed in the future, for example when hovering over a link, but before the user actually clicks on it. This allows to immediately render images when the user decides to navigate to the linked page without any additional network latency.

2.3.1 Impact of Images and Bandwidth Usage on Browsing Experience

Multiple studies demonstrate a strong correlation between webpage load times and user satisfaction. Research shows that even a minor increase in page load time correlates with an increased number of users leaving the website [14]. Users perceive delays exceeding 1 second as disruptive to their cognitive flow [62]. Every other person expects a page to load in less than 2 seconds [84], and 53% of mobile site visits are abandoned if pages take longer than 3 seconds to load [19]. Consequently, **the data volume contributed by unoptimized images directly translates to increased page load times**, potentially exceeding acceptable thresholds, and as a result, contributing to user frustration and attrition.

[8] https://developer.mozilla.org/docs/web/performance
[9] Client-side on-demand image loading only when, or immediately before, they become visible.

Secondly, we can provide different clients with optimized assets based on their unique characteristics. Devices like mobile phones have smaller screens, rely on variable wireless networks, and have limited battery, unlike desktop computers. Therefore, delivering smaller, lower-resolution images to mobile conserves bandwidth and battery, while larger images can be sent to desktops without impacting performance. Network bandwidth is not a uniformly distributed across web users [35]. We access the web across a spectrum of network conditions, from high-capacity wired connections to congested mobile networks. Mobile browsing may be reliant on cellular networks with limited infrastructure, or constrained bandwidth during usage peaks. In such situations, the impact of unoptimized images is amplified, resulting in unnecessary long content load times and a degraded browsing experience.



Screen	Small, limited by portability	Large, typically stationary
Networking	Highly variable, typically wireless	Reliable, wireless or wired connection
Compute	Limited battery power & cooling	Efficient cooling for sustained load

Figure 2.2: Device characteristics and their impact on image optimization

Lastly, developers may not test their applications under constrained network conditions, leading to a lack of awareness of the performance issues faced by users in these scenarios or lower-end devices [74]. For e-commerce platforms, page load speed directly impacts conversion rates and revenue generation [84]. In regions with usage-based internet billing, excessive bandwidth consumption due to unoptimized images can lead to increased data charges for users, creating a financial barrier to accessing web content [29].

2.3.2 Deployment Models of Image Optimization Solutions

The image optimization from the title of our thesis is the practical solution addressing the challenges described in the previous section. Similar to other computing workloads, image solutions can be deployed in on-premises environments or using one of the cloud-deployment models described in section 2.2.

Dedicated Software as a Service for image management, such as Cloudinary[10] or ImageKit[11], offer image resizing, reformatting, cropping, and more. They can operate as standalone services, using providers' public APIs and infrastructure shared between multiple SaaS users up to application layer, or consumed in Backend as a Service model, usually through SDKs. Pricing is based on media-specific operations, such as *image transformations*, storage and bandwidth used for media assets. This can simplify usage bill and operations, as transformations are billed per unit, not compute seconds or consumed

[10] https://cloudinary.com
[11] https://imagekit.io

memory, and these service providers have vital interest in building services that is accessible, and straightforward to integrate with any web application. On the other hand, this can create a dependency: reliance on each next third-party service introduces another potential point of failure, and may limit customization options compared to any self-managed solution. These services are also known as *Image CDNs*[12], because they can leverage distributed infrastructure to cache optimized images at edge locations to minimize latency and improve image performance.

General-purpose CDNs, such as Akamai[13] or Cloudflare[14] also offer products for image manipulation as part of their offering. While the technical implementation is similar to the Image CDNs, we classify these offerings more as a Platform as a Service, usually part of wider offerings that include DNS management, edge compute, web application firewall and traffic scrubbing services. Image optimization is therefore part of more holistic content delivery and security platform, rather than a standalone service.

Open-source image optimization solutions exist, with notable examples including Thumbor[15] - written in Python, using the Pillow[16] library under the hood, imagor[17] - written in Go, using the libvips[18] library. Additionally, we also note extensions to open-source web frameworks, such as the Image Component[19] in Next.js, using sharp[20] under the hood. These projects, while focus on images, include additional features, namely HTTP server functionality (Thumbor and imagor) or UI framework integrations (Next.js). Because of these additional features, these projects are typically not deployed as isolated FaaS functions, especially when core HTTP support is already provided within the built-in runtime in the case of FaaS ZIP deployments.

Instead, **open-source image-manipulation libraries**, such as aforementioned Pillow and sharp, are great candidates to deploy in serverless environments with managed runtimes. They offer developer-friendly interface for collections of lower-level, highly optimized libraries, such as *libjpeg*[21], *libwebp*[22] or *libvips*. They also ensure compatibility between CPU- and OS-dependent library binaries, and the scripting (high-level) programming languages, such as Python or JavaScript. Finally, by offloading request handling to the FaaS provider and image manipulation to an external dependency, serverless function can focus on the actual fine-tuning of the solution: defining allowed quality values and optimization resolutions, supported formats, and performance tuning. Virtually any application-level changes are possible, while the serverless platform takes responsibility for the on-demand resource scaling and complete management of the underlying hardware.

[12] https://web.dev/articles/image-cdns
[13] https://techdocs.akamai.com/ivm/reference/concepts
[14] https://developers.cloudflare.com/images/transform-images/make-responsive-images
[15] https://www.thumbor.org
[16] https://python-pillow.github.io
[17] https://github.com/cshum/imagor
[18] https://libvips.org
[19] https://nextjs.org/docs/app/api-reference/components/image
[20] https://sharp.pixelplumbing.com
[21] https://libjpeg.sourceforge.net
[22] https://chromium.googlesource.com/webm/libwebp

3 System Design

In this chapter, we present objectives and the high-level architecture of the benchmark. We will describe different input parameters that we have considered while designing the image optimization benchmark, and the selection process we used to choose the distinct values of these parameters. It was necessary to focus our study on specific parameters only, as there are too many parameters that can be configured while building a general-purpose image optimization system. We discuss different performance-related metrics we are able to collect during image optimization process in the chosen environments, as well as the methodology used to process these metrics.

3.1 Benchmark Design

The benchmark is composed of two main components:

- **Workload Generation Stack**, which is responsible for generating the image optimization workload and saving the results of the optimization requests in a database.
- **Image Optimization Stacks**, which are provider-specific systems under test (SUTs) that are responsible for hosting and exposing image optimization serverless functions.

Both workload generation and image optimization were implemented in a serverless fashion. For simplicity, workload generator infrastructure was all deployed in one environment - in our case it was AWS, but any cloud provider (or even no cloud infrastructure) could be used, as long as it is connected to the Internet, capable of sending HTTP requests on a schedule and storing the results in a database.

The figure 3.3 shows the main steps of the benchmark:

1. **Event scheduler** invokes the workload generator functions,
2. **Workload generators** send requests to specific image processing functions,
3. **Workload generators** save image optimization results in the database,
4. **Benchmark results** are exported as JSON to a bucket for later offline analysis.

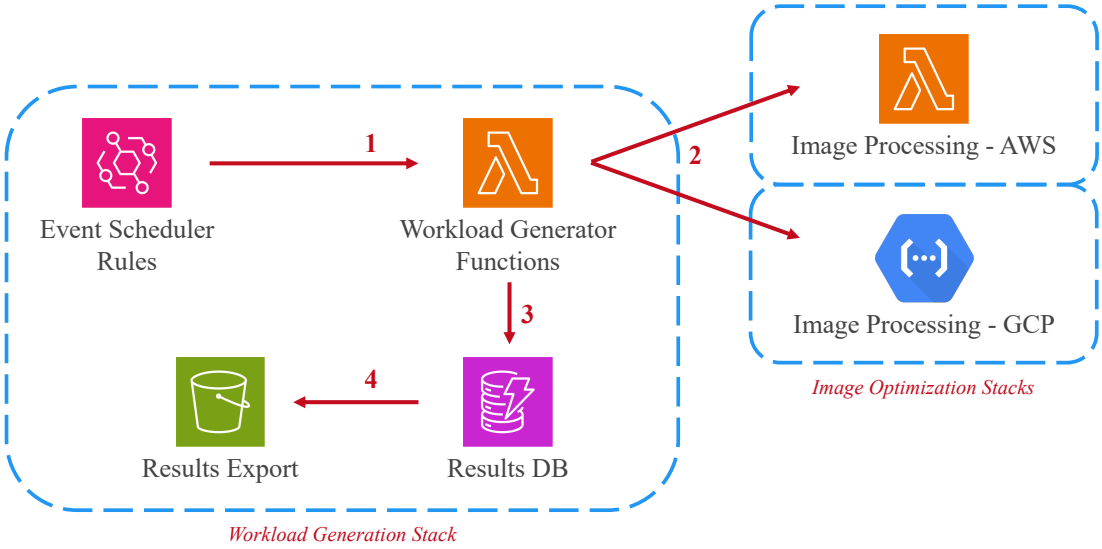


Figure 3.3: High-level design of the image optimization benchmark.

The image optimization stacks were deployed in two different clouds - which we call *providers* - AWS and GCP. These stacks are responsible for providing HTTP endpoints and function invocation in response an incoming HTTP request. Since the optimization process is consistent[23] enough for our purposes, functions only return the metadata of an individual optimization execution, and the output images are discarded.

The image processing function, except for the actual processing, is parsing environment variables, loading image into the function's memory, and calculating the time elapsed since the cold start, as well as the duration of the actual image processing. Figure 3.7 illustrates a simplified function code, outlining the key steps from the environment setup to image processing and response generation.

In the actual benchmark code [95], we do return a few more optimization properties in the response object, omitted in the above code snippet for brevity. Please note that the image processing duration strictly relates to the actual image processing: image resizing, and optionally image reformatting, but does not include any earlier steps necessary to facilitate processing, any code running before the handler initialization, any time spent on loading the image into the function's memory, or request and response handling.

Please also note that in a real-world image optimization scenario, the deployment package of a serverless function does not include any image. An additional network call, or a cloud SDK operation, is required on every function call to retrieve the original, unoptimized image from an external object storage. In our study, we are not interested in measuring performance variability of such operations, and in the interest of cloud service usage minimization we have decided to load the image into memory only once, outside the handler function definition, on a cold start.

3.1.1 Providers, Images, and Optimization Parameters Selection

There were 5 variables for each optimization function variant, presented on figure 3.4. There were in total 108 ($2^2 \cdot 3^3$) image optimization functions deployed, with every variant deployed as separate function to minimize interference between resources. Each function includes the request handler, dependencies required for image optimization, as well as one unoptimized image within its deployment package (ZIP file).

Providers. We have selected Lambda because of its presence in research, as well as its market share [25]. Cloud Run functions (2nd generation) is the new generation of the serverless platform from Google Cloud that includes improved concurrency management, and granular memory and compute configuration. We have selected this platform, as it is still relatively new (2022) offering on the Google Cloud Platform, and many studies only researched its older version.

Input images. To simplify function code and allow for just one optimization run per function invocation, we opted for large input images. Specifically, we selected two high-resolution images produced using DSLR cameras, one in each orientation. For this purpose, in September 2024, we visited an online stock image portal to find popular stock image categories[24]. From the list of most popular searches, two most popular categories, namely *Travel* and *Summer*, were chosen. One image from each category was downloaded from another stock image portal because of the previously mentioned requirement of high

[23] By *consistent* we mean producing comparable metadata outputs for identical input and optimization parameters, sufficient for reliable benchmark comparisons. Strict bit-for-bit determinism is not implied.

[24] https://pexels.com/popular-searches

resolution. For a study focused purely on input images, a larger image dataset, such as [82] would have been used. In our case, only two images were selected in total, because our research was driven by the multidimensional parametrization of the optimization process.

Output formats. Three output formats were selected: JPEG (implies no reformatting), WebP, and AVIF. The latter two represent advancements in compression and browser support over the last decade, but on the other hand they are more computationally demanding[25]. We considered, but eventually did not select JPEG XL due to its limited browser support and custom requirements around for the optimization tooling setup.

Output widths. Three output widths, 640, 1080, and 1920 pixels, were chosen to investigate the impact of this parameter on optimization performance variability. These values represent common display resolutions for mobile, tablet, and desktop devices, respectively. In production environments, more granular or freely configurable width options are commonly implemented, however, too many options can lead to a decreased cache hit ratio, as the frequency of identical requests is reduced. Therefore, in practice, a balance between resolution granularity and cache efficiency should be considered.

Performance settings. As presented earlier in section 2.2.3, the amount of assigned memory and compute power to serverless functions is a recurring theme in prior studies on serverless performance variability. The novel part of our research is that its influence will be evaluated in the practical image optimization scenario, and *not a narrowly focused test* of a predominantly synthetic, CPU/IO/memory-bound nature. Specifically, we used three settings: 885, 1769, and 3538 MB[26] of assigned memory. We have selected these values, because while Google Cloud Run functions let their users configure the amount of associated memory and compute independently, AWS Lambda allocates CPU power in proportion to memory. We have then configured functions have access to half of a vCPU at 885 MB, exactly one vCPU at 1769 MB, and two vCPUs at 3538 MB of memory. Especially with the last two options, we aimed to understand the degree to which the image optimization workload can leverage parallel processing, and whether increasing compute allocation over one vCPU will influence performance variability.

Parameters	Keys	Details
Providers	aws gcp	AWS Lambda x86 Google Cloud Run functions (2nd generation)
Input images	landscape portrait	JPEG, 23.2MB, 8256 × 5504 pixels, 45MP JPEG, 9.9MB, 3456 × 5184 pixels, 18MP
Output formats	jpeg webp heif	JPEG WebP AVIF
Output widths	640 1080 1920	640 pixels 1080 pixels 1920 pixels
Performance settings	885 1769 3538	low, 885 MB memory, 0.5 vCPU medium, 1769 MB memory, 1 vCPU high, 3538 MB memory, 2 vCPU

Figure 3.4: All deployed image optimization function variants.

[25] https://developers.cloudflare.com/images/transform-images/#format-limitations
[26] AWS Lambda is using the abbreviation MB (rather than MiB) to refer to 1024 KB.

3.1.2 Regions, Runtime, and Image Optimization Library Selection

In this section, we will describe the parameters that remained constant across all functions. Each additional dimension results in an exponential increase in the number of functions and the volume of collected data. We considered adding further variables, but eventually we decided against it to limit the complexity of the multidimensional data analysis and data visualization.

Cloud regions. For the image optimization stacks, cloud regions in close geographical proximity were selected: AWS region in North Virginia (`us-east-1`) and GCP region in South Carolina (`us-east1`). North Virginia is the oldest AWS region, and was selected for consistency and comparability with existing literature, such as [33]. This fact also guided the region choice for the GCP deployment. In production environments, where latency minimization is key, image optimization stacks can be deployed in multiple physical locations, and latency-based or location-based routing can be used to route users to the closest region.

Runtimes. Another property that we haven't parametrized was *runtime*. We have selected the built-in Node.js runtimes in the latest stable version (22) at the time of writing. This was an intentional choice to illustrate that built-in runtimes can be an effective approach to tackle image optimization, and that building a custom container image is not required, even if using OS-specific and CPU-specific dependencies is required. Secondly, the very choice of the programming language could be parametrized, but we have decided that this is out of this benchmark's scope. Thirdly, there may exist alternative implementations of runtimes (such as Deno[27], PyPy[28], GraalVM[29], and more). However, these are not natively provided by the serverless providers, not compatible with ZIP deployments, and therefore out of this benchmark's scope.

Image optimization library. For image optimization, we chose the sharp Node.js module. It leverages libvips, a performant image processing library, known for its multi-threading capabilities and efficient memory management [17]. Using libvips gives sharp performance of a native C library, which is advantageous in any scenario, not only serverless. We have reviewed capabilities (image format support), popularity (weekly npm downloads), and performance [31] of sharp's alternatives, including ImageMagick[30], Jimp[31], and squoosh[32], and ultimately decided to implement our benchmark exclusively with sharp. Figure 3.5 shows differences in image manipulation performance between libvips and similar image manipulation libraries that supported our decision.

While we have decided not to proceed with parametrization, we would like to acknowledge that the categories presented above would be a valuable addition to any benchmark not specifically requiring high-frequency measurements. For example, in the compatibility benchmarking of different cloud service providers, runtimes, software libraries, and data formats, adding more dimensions could be beneficial. This is because, unlike the requirements of this thesis, a lower frequency of benchmarking would be sufficient given the consistent long-term behavior of qualities such as compatibility.

[27] https://deno.com
[28] https://pypy.org
[29] https://graalvm.org
[30] https://imagemagick.org
[31] https://jimp-dev.github.io/jimp
[32] https://squoosh.app

Software	Run time (secs real)	Memory (peak RSS MB)	Times slower
libvips 8.14 Lua	0.37	76	0.97
libvips 8.14 C/C++	**0.38**	**84**	**1.00**
libvips 8.14 PHP	0.41	103	1.07
tiffcp	0.45	581	1.18
libvips 8.14 Python	0.47	133	1.24
libvips 8.14 Ruby	0.49	107	1.29
libvips 8.14, JPEG images	0.98	177	2.57
libvips 8.14 CLI	1.08	83	2.84
Pillow-SIMD 9.0.0	1.1	985	2.89
GraphicsMagick 1.4	1.42	1982	3.73
libvips 8.14, one thread	1.49	52	3.92
nip2	1.59	158	4.18
GEGL 0.4.38-1, JPEG images	6.13	751	5.0
libgd 2.3.3-6, JPEG images	4.9	1040	5.00
NetPBM 1.13	1.70	289	5.15
ImageJ 1.53	2.59	542	6.82
OpenCV 4.6	2.85	791	7.5
rmagick 6.9.11-60	2.87	2785	7.55
OpenImageIO 2.3.18	2.91	2223	7.65
convert 6.9.11-60	2.99	2002	7.86
imwand.py 6.9.11-60	3.08	2015	8.11
ExactImage 1.0.2-9	3.21	475	8.44
Imlib2 1.7.4-2	3.39	1018	8.92
FreeImage 3.18.0	4.21	752	11.08
imagick 6.9.11-60	6.61	2018	17.39
Pike 8.0.702	5.90	1376	17.88
GMIC 2.9.4-4	9.87	2995	25.97
Octave 7.2.0-1	27.08	6505	71.26

Figure 3.5: Image processing libraries performance comparison involving TIFF loading, cropping, scaling, and sharpening after [17].

3.2 Implementation

To run our benchmark at scale, we required a way to automatically generate workloads and simulate user requests over time. We used a serverless event scheduler to trigger Workload Generator functions every 5 minutes[33]. These functions are responsible for sending requests to the image processing functions under test.

Our benchmark uses a simple and predictable path-based routing method to direct requests to the correct function, which makes it easy to target specific image optimization function. To simplify domain name management, we grouped functions from the same cloud provider under a single domain name. This is how Cloud Run works by default: functions in the same project and region share the base domain. However, Lambda Function URLs do not, and require more manual effort for the same result. We have added a (cloud-native) proxy to route requests to correct Lambda functions based on the path. Amazon API Gateway is one of the AWS services that can fulfill this purpose, but there are various other ways to solve this slight operational inconvenience. In the benchmark, we never transfer images over the network, meaning any potential restrictions in HTTP body size most likely won't cause any problems during benchmark's execution. However, serverless image optimization, especially with large input images and low amount of assigned compute, may be prone to timeouts on various layers of the serverless benchmark stack. Adding any new element to the benchmark architecture needs to be done carefully, so that it does not skew results, whether quantitative (such as the number of measurements) or qualitative (response latency, and the actual measurements embedded in the response body). In this case, the benchmark does not measure the total response latency, and it is therefore safe to use additional proxy for the operational benefit of simpler domain management.

The path-based routing is important because it allows the workload generator to work across different providers. It relies on standard HTTP requests that do not utilize any additional request characteristics, like cookies, headers, query strings, or request body. Workload generation does not rely on any proprietary interface or features of a particular serverless provider. While we are focused on serverless environments, this approach could also work with traditional, serverful image processing, or any other workloads, if required.

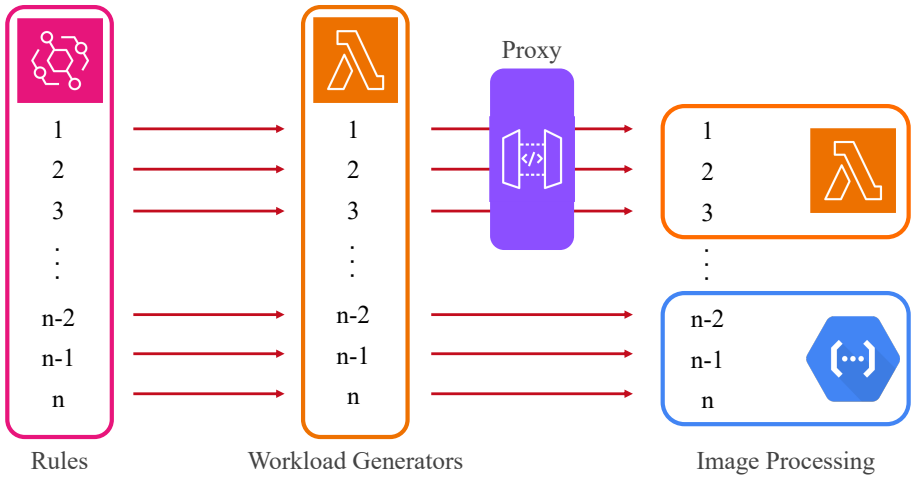


Figure 3.6: 1:1:1 mapping between rules, workload generators, and image processing functions, and proxy providing single domain name for Lambda functions.

[33]See Figure 3.3

3.2.1 Results Collection and Data Analysis

To enable thorough performance analysis, we designed the image processing functions to return detailed metadata upon each function invocation. This metadata, structured as a JSON object, includes performance indicators and additional contextual information.

The specific properties addressing our research questions about FaaS performance variability in the image optimization scenario are:

- *coldStart* flags the first invocation in a new execution environment,
- *durationProcessingMs* captures the time spent performing single image optimization,
- *durationSinceColdStartMs* measures the elapsed time from the initial cold start of the execution environment to the start of image processing,
- *environmentId* is a unique identifier generated at cold start to track executions within the same environment during the whole lifetime of the environment,
- *handlerCount* tracks the number of invocations handled by a specific environment instance, and is incremented with each request / function invocation,
- *imageName* indicates whether the landscape or portrait image was processed,
- *providerArch* indicates the processor architecture of the execution environment,
- *providerCpus* reports the number of CPU cores available to the function,
- *providerMemoryAssociated* (MB) reports the configured memory size for the function,
- *providerMemoryAvailable* (MB) reports the total available memory,
- *providerMemoryUsageRss* (MB) measures the memory usage of the function process,
- *providerName* identifies the image processing stack provider,
- *providerNodeVersion* indicates the Node.js version embedded into the runtime.

```
const coldStartTime = Date.now();                                    // set cold start timestamp
const environmentId = crypto.randomUUID();                           // generate stable environment ID
const format = process.env.OUTPUT_FORMAT!;                           // load image format from env
const width = Number(process.env.OUTPUT_WIDTH!);                     // load image width from env
const image = readFileSync('/opt/nodejs/image.jpg');                 // load image from disk into memory

let handlerCount = 0;                                                // initialize handler counter
const processor = sharp(image).resize(width).toFormat(format);       // set up image processing

export const handler = async () => {
  handlerCount++;                                                    // increment handler counter

  const processingStartTime = Date.now();                            // set current timestamp
  await processor.toFile('/tmp/sharp_out');                          // process image
  const durationProcessingMs = processingStartTime - Date.now();     // save processing duration

  return JSON.stringify({
    coldStart: handlerCount === 1,                                   // true on cold start only
    durationProcessingMs,                                            // image processing duration
    durationSinceColdStartMs: processingStartTime - coldStartTime,   // duration since environment start
    environmentId,
    handlerCount
});
```

Figure 3.7: Simplified image processing function logic.

For optimization consistency monitoring, we also return additional metadata related to images before and after their optimization, such as their byte size, formats, and resolution. Values reported by these parameters do not change over time for the individual optimization configurations. All results are stored in a serverless, non-relational database managed by the cloud provider, where the workload generation stack is deployed. Upon getting the response from image optimization function, the workload generator adds a timestamp field and stores remaining data *as is* in the database using the cloud provider's SDK.

Because benchmarking results are processed off-cloud, one database table is shared across all workload generator functions to simplify the data export procedure. The database we used, DynamoDB[34], offers an export in a chunked JSON format. After conversion to CSV it can be consumed by standard data analytics tooling, and we have used pandas[35] library to clean and aggregate benchmark data and the seaborn[36] library for data visualization.

3.2.2 Serverless Benchmark Deployment

Implementation of functions across providers is as similar as possible. Managed runtimes for Lambda and Cloud Run functions differ in only two meaningful ways for the image optimization scenario.

Firstly, the location of the image within the deployment package is provider-specific. On AWS, it is possible to use so-called Lambda layers[37], and the selected unoptimized image is mounted in the Lambda file system. The benefit of this approach is that there is only one piece of image optimization code, one function definition to maintain. With Cloud Run functions, we needed to build two separate variants of the ZIP deployment packages, one containing landscape image, and the other containing portrait image.

Secondly, the platforms differ in the exact way how function handlers are exposed. On Lambda, the function entry point has to export a callable function, usually called *handler*. The name of the function can be customized and in such case needs to be consistent both in the function code and the configuration of the function. On Cloud Run functions, the entry point needs to import a JavaScript library provided by Google Cloud called Functions Framework[38]. Instead of an export, the *http* function available in the framework library needs to be called from the function code.

In both cases, processing relies on the fact that the provider is responsible for invoking an entry point upon an event, such as an HTTP request. This abstraction allows the image optimization logic to be consistent across providers, with the only difference being the package organization and handler format. The underlying environments, while implemented with some differences, provide a comparable set of capabilities, enabling the deployment of functionally equivalent image optimization across both platforms. This facilitates portability, reducing the effort when transitioning between providers in real-world, production scenarios.

As both workload generator and image optimization functions were using Node.js runtime, we also wanted to use an imperative Infrastructure as Code (IaC) framework to deploy the stacks onto serverless platforms. We acknowledge the popularity of declarative approach,

[34]https://aws.amazon.com/dynamodb
[35]https://pandas.pydata.org
[36]https://seaborn.pydata.org
[37]https://docs.aws.amazon.com/lambda/latest/dg/chapter-layers.html
[38]https://cloud.google.com/functions/docs/functions-framework

including Ansible[39], HashiCorp's Terraform[40], and OpenTofu[41]. Nevertheless, platforms' unique requirements described earlier, and the additional proxy module in the AWS case, made the initial goal of a unified, provider-agnostic approach less advantageous than anticipated. As a result, we decided to optimize the deployment strategy for AWS - the provider where the workload generation was implemented. We have reviewed various imperative IaC solutions, including Cloud Development Kit for Terraform[42], Pulumi[43], and Wing[44]. Eventually, we have decided to use AWS Cloud Development Kit[45] (CDK), as it offers a dedicated Node.js Lambda function *construct*, which is a reusable component that encapsulates cloud infrastructure definition. Similar to other IaC solutions, CDK provides a library of constructs covering most of the commonly used AWS services and features.

AWS CDK supports multiple programming languages, including TypeScript, JavaScript, Python, Java, C#, and Go. As an abstraction layer over AWS CloudFormation[46], CDK allowed us to use usual loops and variables instead of declarative and relatively verbose YAML or HCL templates. The use of nested loops to iterate through various optimization parameters was extremely useful, making the deployment configuration far more concise.

For Cloud Run functions, we prepared shell scripts that build and deploy all function variations with correct optimization parameters set as the environment variables and correct memory and compute settings. This approach, while less elegant, turned out to be very effective, and ultimately introduced only minimal operational overhead. The benchmark code was written and deployed around November 2024, and is using the service offerings and feature sets of both service providers available at that time.

```
// Define and iterate over benchmark parameters
const providers = ['aws', 'gcp'] as const;
const imageNames = ['landscape', 'portrait'];
const memorySizes = [885, 1769, 3538]; // 0.5vCPU, 1vCPU, 2vCPU
const outputFormats = ['avif', 'jpeg', 'webp'];
const outputWidths = [640, 1080, 1920];

providers.forEach(provider => {
  imageNames.forEach(imageName => {
    memorySizes.forEach(memorySize => {
      outputFormats.forEach(outputFormat => {
        outputWidths.forEach(outputWidth => {
          const id = `${imageName}-${memorySize}-${outputFormat}-${outputWidth}`;
          const workloadGenerator = new WorkloadGeneratorFunction(provider, id, resultsTable);

          new Rule(`Rule-${cloud}-${id}`, {               // periodically trigger workload generation
            ruleName: `rule-${cloud}-${id}`,              // set unique rule name
            schedule: Schedule.rate(Duration.minutes(5)), // set desired interval as needed
            targets: [workloadGenerator]
          });
        });
      });
    });
  });
});
```

Figure 3.8: CDK semantics allow for an extremely compact infrastructure definition.

[39]https://github.com/ansible/ansible
[40]https://terraform.io
[41]https://opentofu.org
[42]https://developer.hashicorp.com/terraform/cdktf
[43]https://pulumi.com
[44]https://winglang.io
[45]https://aws.amazon.com/cdk
[46]https://aws.amazon.com/cloudformation

4 Results

This chapter presents the results from our two-month-long benchmark of image optimization, focusing on performance metrics collected across two serverless platforms, and 54 image optimization configurations per platform. Instead of optimized images, the image optimization functions described in Section 3.2.1 return a handful of characteristics, such as the cold start flag, image processing duration, and optimization invocation time. The measurements cover a period of 62 days between December 2024 and February 2025.

4.1 Missed Executions

Missed execution is a situation when we expect a response from the image optimization function and a corresponding record in the database, but the number of recorded observations is smaller than expected. The benchmark does not explicitly record such situation in any way, however, any error occurrences can be reasoned about by the observation of a lower than expected number of data points for a given time frame and a set of optimization parameters. For example, when surveying the results for specific function variant over a duration of one hour, 12 data points can be expected. A lower number of measurements is a very strong indicator[47] of missed executions of that specific functions in the selected period of time. The benchmark does not additionally track the effectiveness of the rule scheduler itself.

From exactly 1928448 scheduled executions in total:

- workload generator collected responses from 1924484 (99.79%) executions,
- workload generator missed responses from 3964 (0.21%) executions,
- 63.94 data points were missed on an average day.

From 964224 scheduled executions per provider,

- benchmark collected 961113 (99.68%) and missed 3111 (0.32%) executions from AWS,
- benchmark collected 963371 (99.91%) and missed 853 (0.09%) executions from GCP,
- AWS failed to deliver responses 3.65x times more often than GCP.

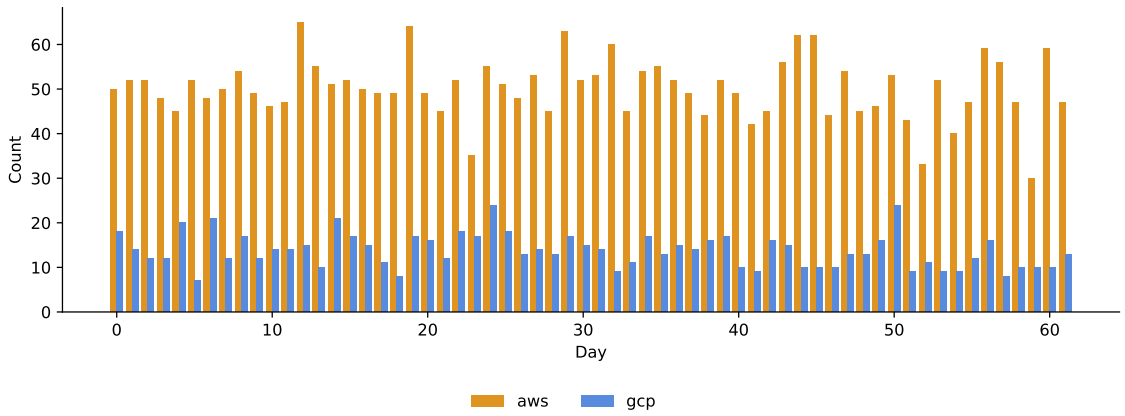


Figure 4.9: Missed executions by provider.

[47] Due to the distributed nature of the scheduler service, there can be a delay of several seconds between the time the scheduled rule is triggered and the time the target service runs the target resource.

4.2 Cold Starts

Cold start latency is not specifically recorded as part of this benchmark. In the serverless image optimization scenario, we exclusively focus on the image optimization latency. Nevertheless, the benchmark is collecting data about an *occurrence* of the cold start. This value could be inferred from the *handlerCount* metric, but the optimization functions are also setting an additional property called *coldStart* for convenience.

From exactly 1924484 collected responses in total:

- workload generator recorded 1878363 (97.60%) warm starts,
- workload generator recorded 46121 (2.40%) cold starts.

From 961113 and 963371 of collected data points from AWS and GCP respectively,

- benchmark recorded 915965 (95.30%) warm and 45148 (4.70%) cold starts on AWS,
- benchmark recorded 962398 (99.90%) warm and 973 (0.10%) cold starts on GCP,
- it was 48.18 times more likely to observe a cold start on AWS than on GCP.

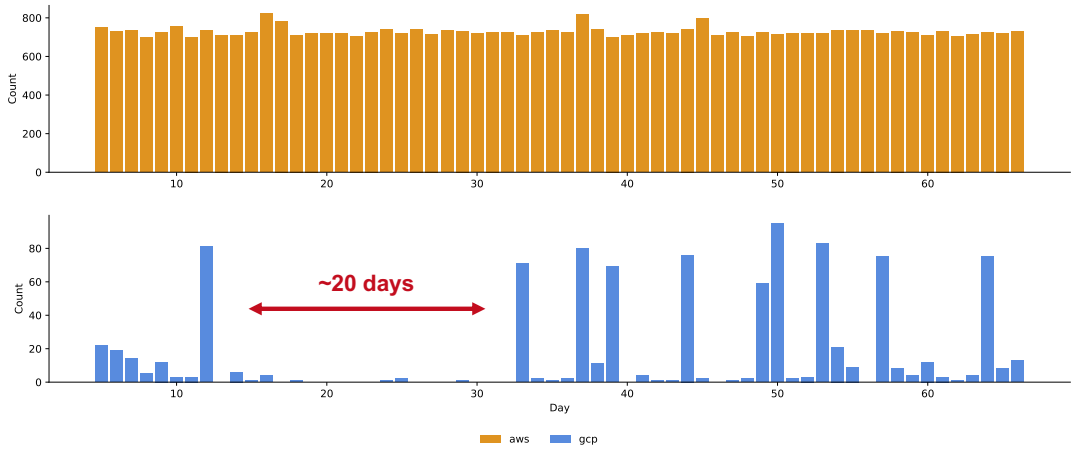


Figure 4.10: Amount of cold starts per day per provider.

4.3 Environment Lifetime and Environment Reuse Metrics

These two metrics are closely related to the cold start phenomenon. Although not specifically targeted by this benchmark, environment reuse can be beneficial to preserve state between function executions, and potentially reduce amount of work that is required before the handler function can be called. The benchmark used two metrics in this context:

- environment lifetime is measured by *durationSinceColdStartMs* metric,
- environment reuse is measured by the *handlerCount* metric.

Provider	p50	p95	p99	p100
AWS	59 minutes	1 hour 59 minutes	2 hours 9 minutes	2 hours 18 minutes
GCP	2 days 17 hours	16 days 17 hours	19 days 18 hours	20 days 16 hours

Figure 4.11: *durationSinceColdStart* distribution - environment lifetime per provider.

Provider	p50	p95	p99	p100
AWS	12	24	27	28
GCP	783	4822	5693	5955

Figure 4.12: *handlerCount* distributions per provider.

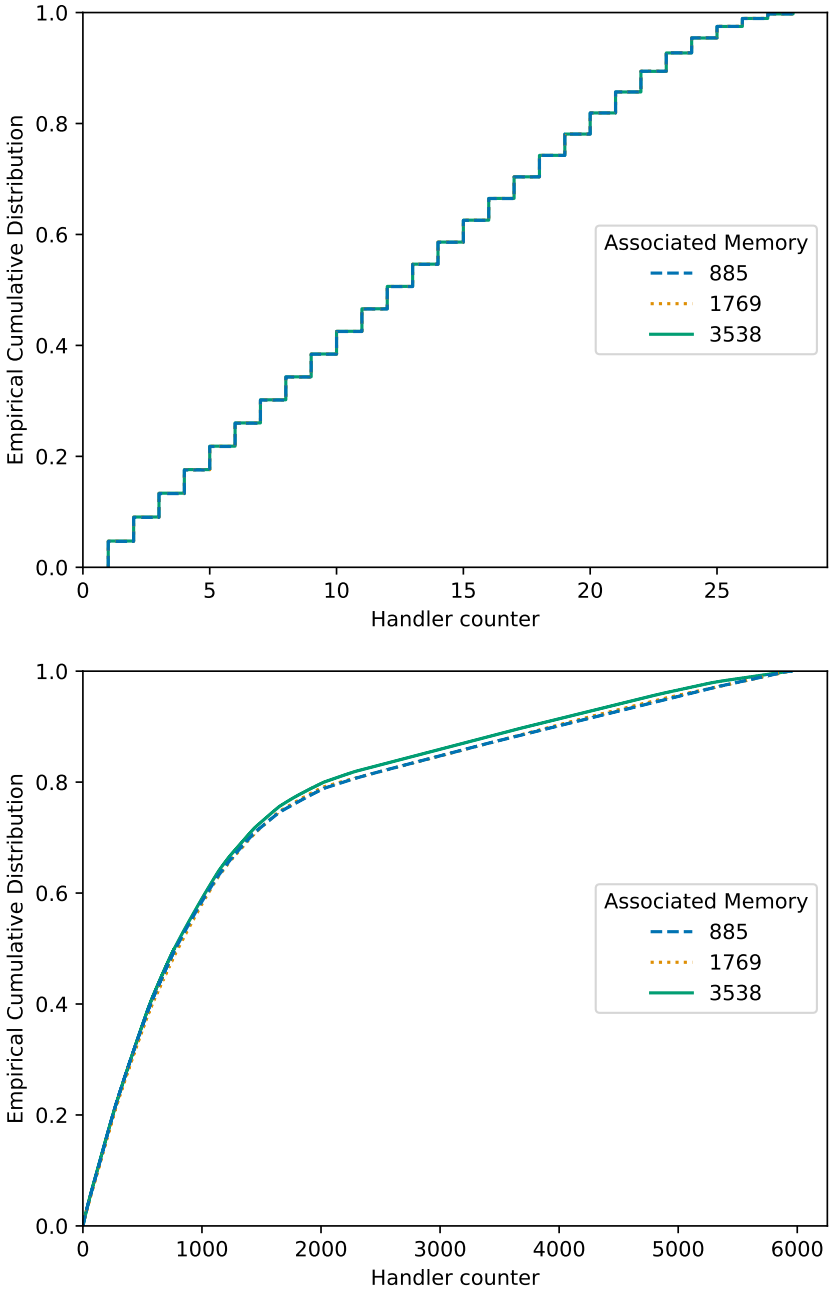


Figure 4.13: Environment reuse on Lambda (above) and Cloud Run functions (below).

The longest running Cloud Run function was reusing the environment over 200 times longer than the longest running Lambda function. Environment reuse **per provider** is consistent across different input images, output formats, output widths and performance settings.

4.4 Measuring Image Processing Duration

This metric determines how quickly optimized images are ready to be delivered to end users. It is measured in millisecond resolution and captures the core operation of transforming images according to specified optimization parameters. It is heavily related to the performance settings, such as amount of compute assigned to the function, output format and width, as well the input image resolution. For completeness, visualization of absolute optimization times is provided for one of the input images in all 54 function variants.

Please note that lower performance settings imply longer processing duration. Different output formats have different computational complexity, with JPEG having the shortest processing times, and AVIF having the longest. Different providers also exhibit difference in processing duration, even for *"the same"* compute (vCPUs) and memory configurations.

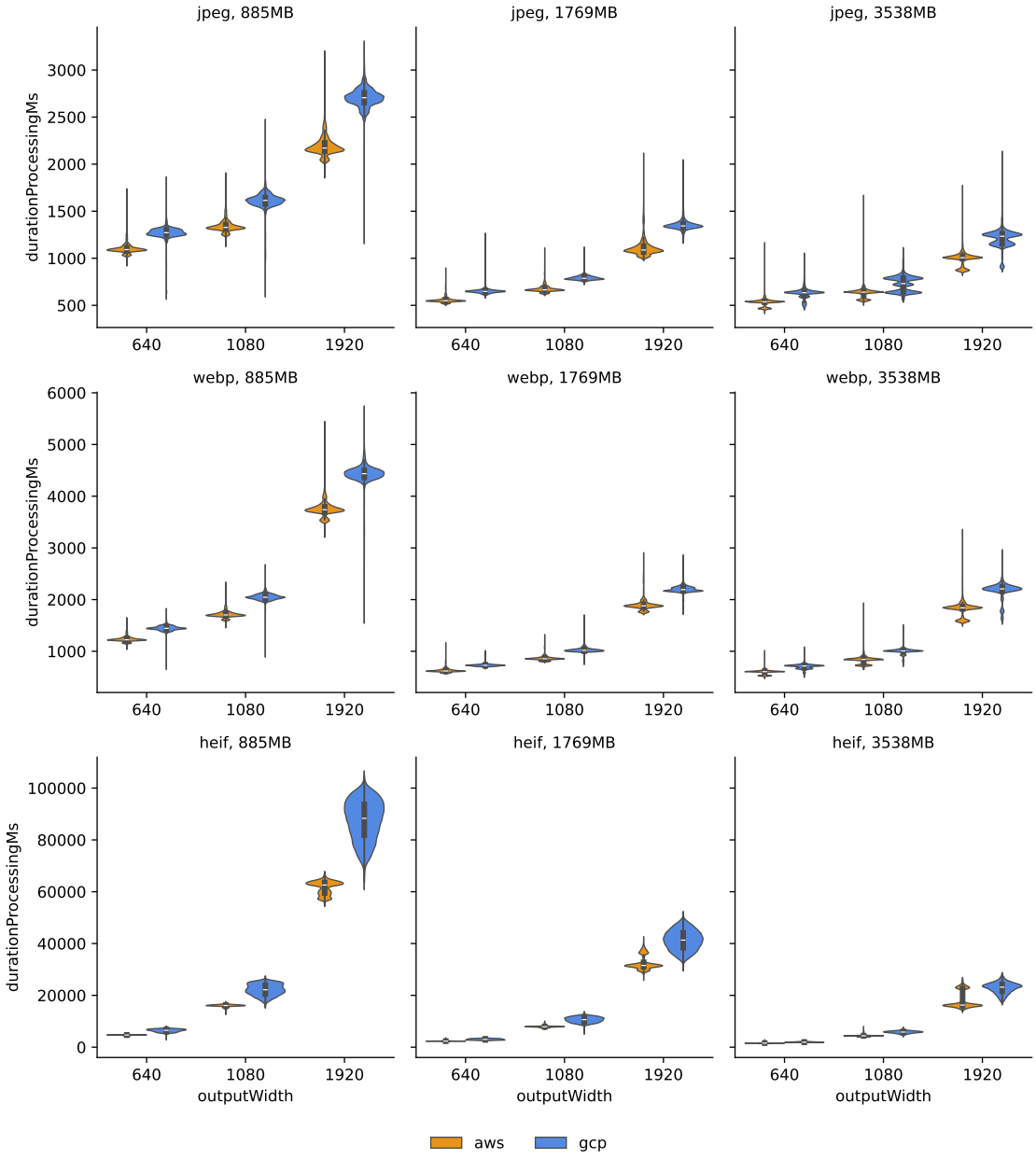


Figure 4.14: Absolute image processing durations *(durationProcessingMs, portrait)*.

4.5 Daily Temporal Variability

The benchmark data shows changes of the processing duration depending on the time of day when the optimization workflow is executed. The variability is the biggest for the AVIF format, and for functions with the most memory assigned for image processing.

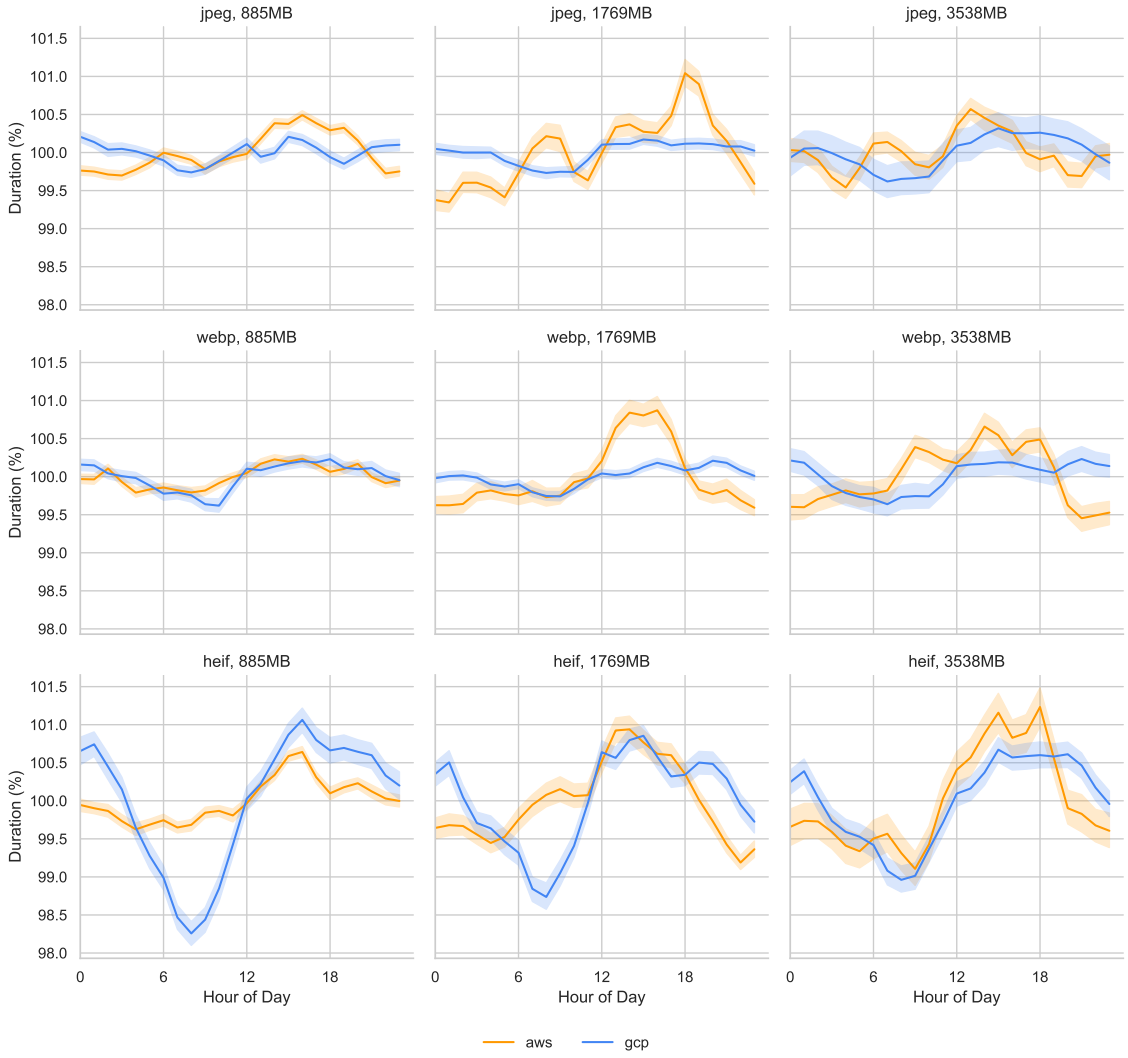


Figure 4.15: Average image optimization duration throughout a day in various configurations.

In addition to the *hourly* variability, another *minutely* variability is present given the aggregation basis of individual minutes instead of hours. The *hourly variability* is the bigger of the two, around 1%. The *minutely variability* is lower, usually closer to 0.5%. This indicates serverless performance changing sharply at the beginning/end of every hour.

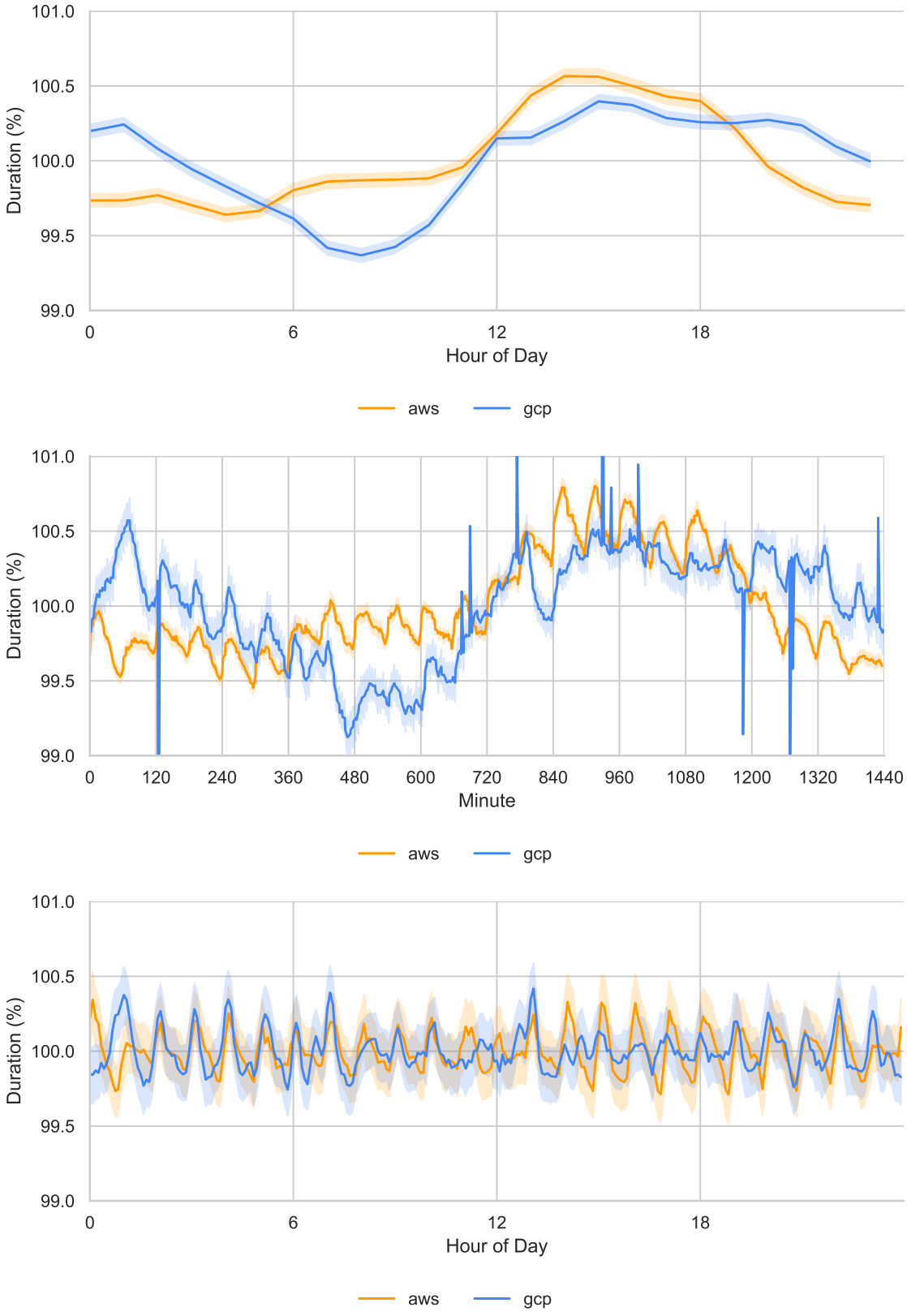


Figure 4.16: Average image optimization duration throughout a day in various aggregations.

4.6 Weekly Temporal Variability

The benchmark data also shows changes of the image processing duration depending on the time of week when the optimization workflow is executed. Here, similar to the previous section, we have performed aggregations based on the output format, as well as performance setting assigned to the function. Also, similar to the daily temporal variability, it can be seen that these values are higher for more computationally complex image formats and higher performance settings.

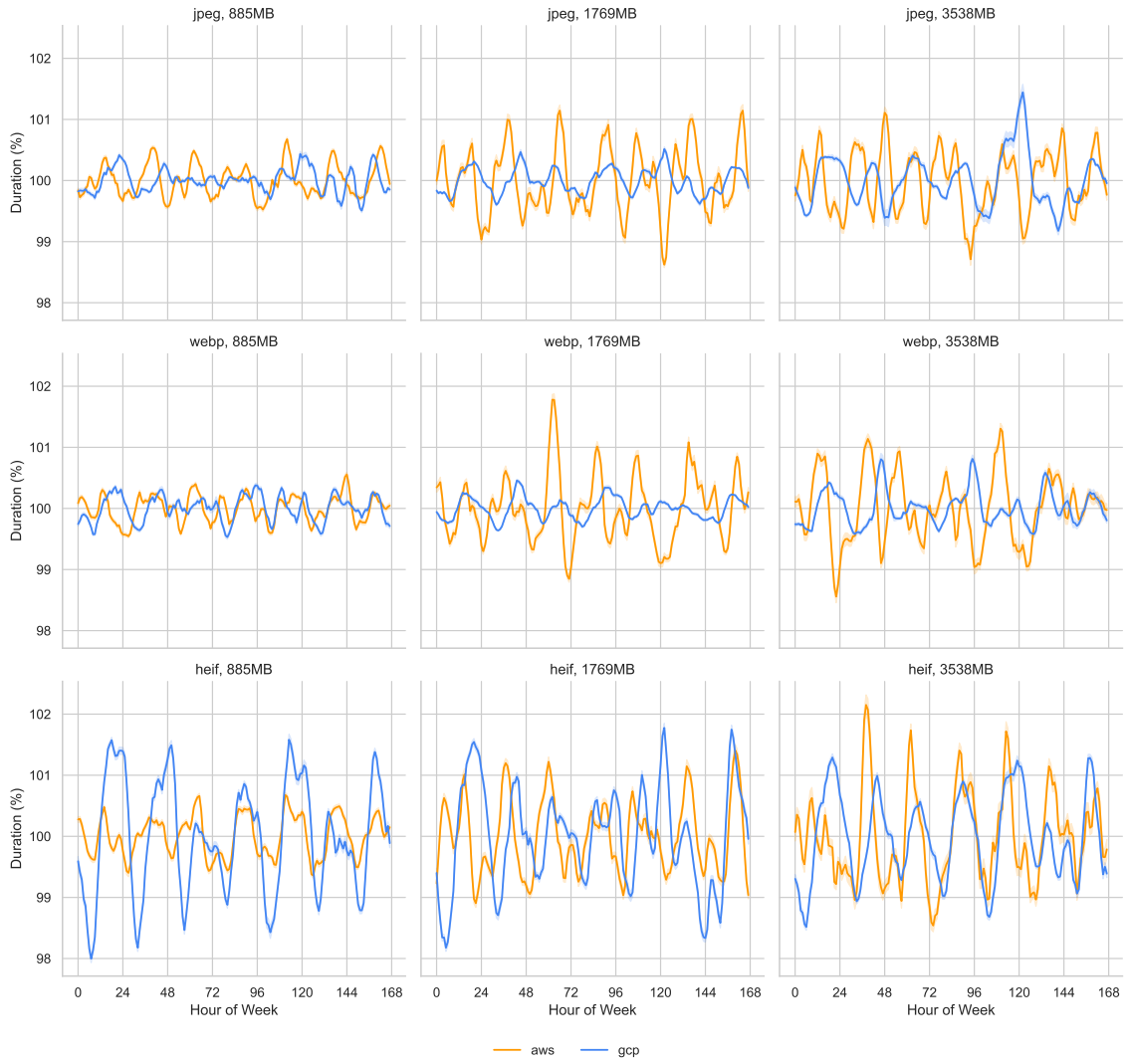


Figure 4.17: Average image optimization duration throughout a day in various configurations.

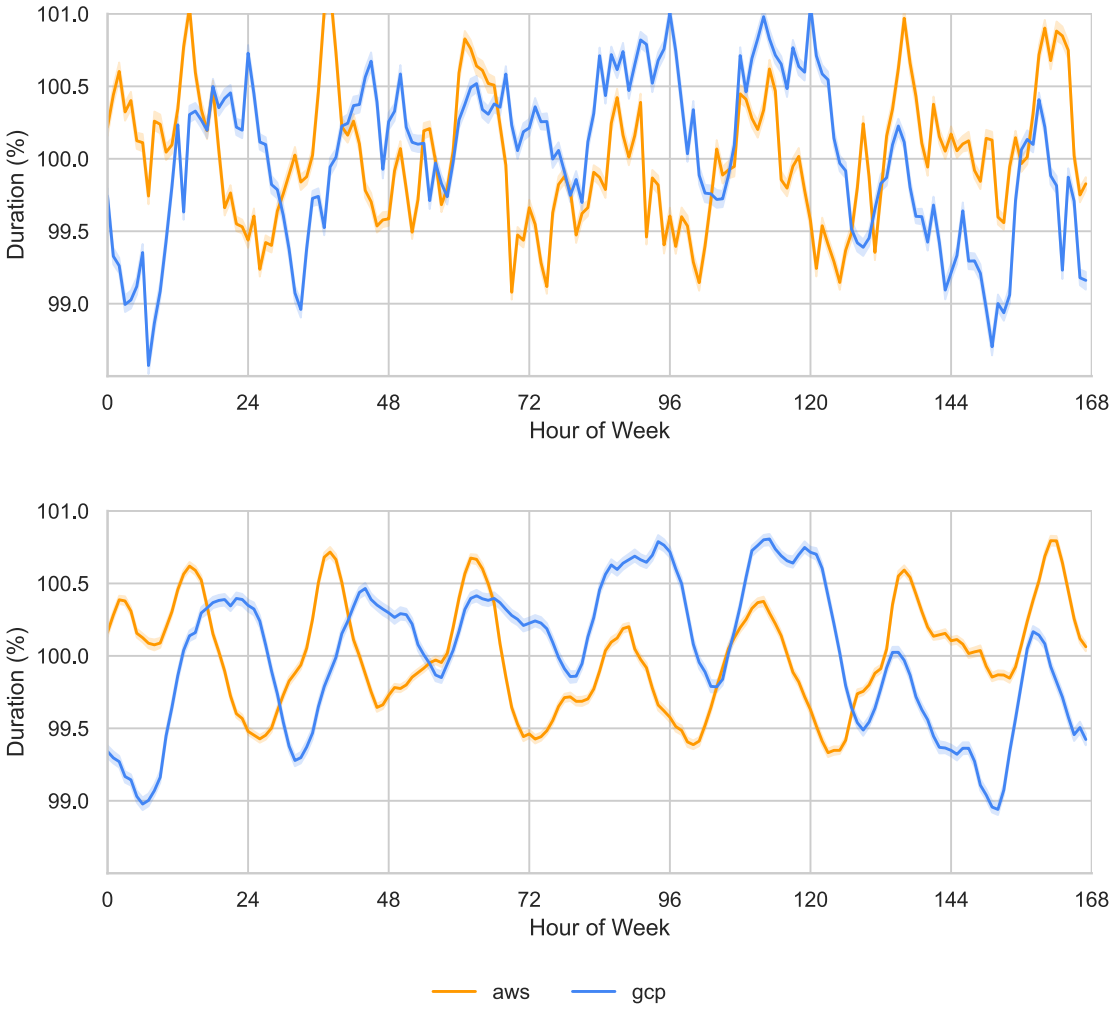


Figure 4.18: Image optimization duration throughout a week: raw aggregated values (above) and smoothened aggregate based on a rolling average (below).

4.7 Environment-related Metrics

These metrics exhibited both static and variable behavior over the total duration of the experiment:

- *providerArch* - x64 across all function variants on both serverless platforms,
- *providerCpus* - variable on AWS, but constant on GCP,
- *providerMemoryAvailable* - variable on AWS, but constant on GCP, these metrics represent the actual vCPU and memory available from within the function,
- *providerNodeVersion* - v20.18.0 across all function variants.

providerName	providerMemoryAssociated	providerCpus	percentage
AWS	885	2	98.73%
		3	0.33%
		4	0.39%
		5	0.16%
		6	0.39%
	1769	2	96.31%
		3	1.65%
		4	1.04%
		5	0.46%
		6	0.55%
	3538	3	94.88%
		4	3.94%
		5	0.63%
		6	0.55%
GCP	885	2	100.00%
	1769	2	100.00%
	3538	4	100.00%

providerName	providerMemoryAssociated	providerMemoryAvailable	percentage
AWS	885	1191.73	92.57%
		1706.73	5.61%
		3214.72	0.15%
		3214.72	0.40%
		5612.16	0.33%
		7489.98	0.39%
		9278.80	0.16%
		10706.62	0.39%
	1769	3214.72	20.90%
		3214.72	75.40%
		5612.16	1.65%
		7489.98	1.04%
		9278.80	0.46%
		10706.62	0.55%
	3538	5612.16	94.88%
		7489.98	3.94%
		9278.80	0.63%
		10706.62	0.55%
GCP	885	1024.00	100.00%
	1769	1687.05	100.00%
	3538	3374.10	100.00%

Figure 4.19: Differences in actual hardware configuration for identical performance setting.

4.8 Handler Count-Related Duration Variability

We have observed increased average image processing duration on the first 3 to 6 optimization invocations. This should be not confused with the *cold start latency*. This is the duration of the individual image optimization instruction within the function handler.

On average, the first execution of the image optimization on AWS Lambda is around 7% higher than baseline. Actually, it is a bit more than that, due to the relative commonness of the cold starts on this platform, and given the p100 handlerCount on AWS Lambda presented in the previous section is only 28. This effect is also noticeable, but not so pronounced on Google Cloud Run functions, where the value reaches only 3% above the *true* baseline. Please note the difference in the confidence intervals between the providers.

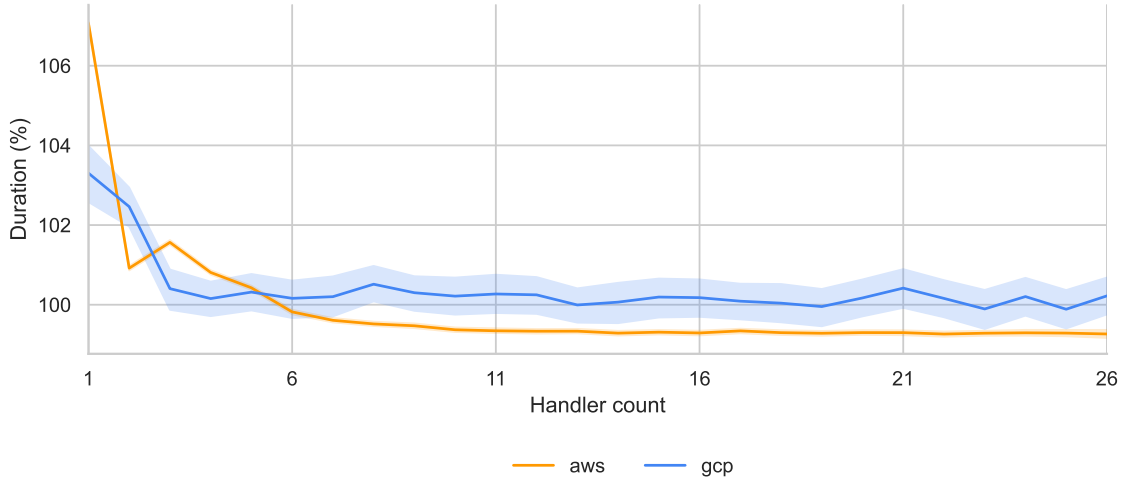


Figure 4.20: Average image processing duration by handler count.

5 Discussion

5.1 Performance variability analysis

5.2 User vs operator perspective

5.3 Cloud computing as a utility

6 Related Work

6.1 Serverless performance research

6.2 Image processing research

7 Conclusion & Future Work

"Three sentences at the end about stuff that didn't work out"

References

[1] Adobe. *Digital Negative (DNG) Specification, Version 1.7.0.0*. 2023.

[2] Nasir Ahmed, T. Raj Natarajan, and Kamisetty R. Rao. "Discrete Cosine Transform". In: *IEEE Transactions on Computers* C-23.1 (1974), pp. 90–93.

[3] Jyrki Alakuijala, Jon Sneyers, Luca Versari, and Jan Wassenberg. *JPEG White Paper: JPEG XL Image Coding System*. 2023.

[4] Luis A. Aranda, Alfonso Sánchez-Macián, and Juan A. Maestro. "A Methodology to Analyze the Fault Tolerance of Demosaicking Methods against Memory Single Event Functional Interrupts (SEFIs)". In: *Electronics* 9.10 (2020), p. 1619.

[5] AWS. "What is Cloud Computing?" In: *AWS Whitepaper: Overview of Amazon Web Services* (2024).

[6] J. Bankoski, J. Koleszar, L. Quillio, J. Salonen, P. Wilkins, and Y Xu. *RFC 6386: VP8 Data Format and Decoding Guide*. 2011.

[7] Jim Bankoski, Paul Wilkins, and Yaowu Xu. "Technical Overview of VP8, an Open Source Video Codec for the Web". In: *2011 IEEE International Conference on Multimedia and Expo*. IEEE. 2011, pp. 1–6.

[8] Pankaj Kumar Bansal, Vijay Bansal, Mahesh Narain Shukla, and Ajit Singh Motra. "VP8 Encoder — Cost effective implementation". In: *SoftCOM 2012, 20th International Conference on Software, Telecommunications and Computer Networks*. IEEE. 2012, pp. 1–6.

[9] Nabajeet Barman and Maria G Martini. "An evaluation of the next-generation image coding standard AVIF". In: *2020 Twelfth International Conference on Quality of Multimedia Experience (QoMEX)*. IEEE. 2020, pp. 1–4.

[10] David Bermbach, Erik Wittern, and Stefan Tai. *Cloud Service Benchmarking*. Springer, 2017.

[11] Tim Berners-Lee, Robert Cailliau, Ari Luotonen, Henrik Frystyk Nielsen, and Arthur Secret. "The world-wide web". In: *Communications of the ACM* 37.8 (1994), pp. 76–82.

[12] Mike Bishop. *RFC 9114: HTTP/3*. 2022.

[13] Matic Broz. *Photo Statistics: How Many Photos are Taken Every Day?* 2024.

[14] Jake Brutlag. "Speed Matters". In: *Google Research Blog* (2009).

[15] Wilhelm Burger and Mark J Burge. *Digital Image Processing: An Algorithmic Introduction*. Springer Nature, 2022.

[16] Y. Collet and M. Kucherawy. *RFC 8878: Zstandard Compression and the 'application/zstd' Media Type*. 2021.

[17] John Cupitt. *Speed and memory use - libvips*. https://github.com/libvips/libvips/wiki/speed-and-memory-use. 2024.

[18] Jaime Dantas, Hamzeh Khazaei, and Marin Litoiu. "Application Deployment Strategies for Reducing the Cold Start Delay of AWS Lambda". In: *2022 IEEE 15th International Conference on Cloud Computing (CLOUD)*. IEEE. 2022, pp. 1–10.

[19] Google Data. *Global, n=3,700 aggregated, anonymized Google Analytics data from a sample of mWeb sites opted into sharing benchmark data*. 2016.

[20] Alma Davenport. *The History of Photography: An Overview*. Unm Press, 1999.

[21] Peter Deutsch. *RFC1951: DEFLATE Compressed Data Format Specification version 1.3*. 1996.

[22] Jarek Duda. *Asymmetric numeral systems*. 2009. arXiv: 0902.0271.

[23] Jarek Duda. *Asymmetric numeral systems: entropy coding combining speed of Huffman coding with compression rate of arithmetic coding*. 2014. arXiv: 1311.2540.

[24] Jarek Duda, Khalid Tahboub, Neeraj J. Gadgil, and Edward J. Delp. "The use of asymmetric numeral systems as an accurate replacement for Huffman coding". In: *2015 Picture Coding Symposium (PCS)*. IEEE. 2015, pp. 65–69.

[25] Simon Eismann, Joel Scheuner, Erwin Van Eyk, Maximilian Schwinger, Johannes Grohmann, Nikolas Herbst, Cristina L. Abad, and Alexandru Iosup. "A review of serverless use cases and their characteristics". In: *arXiv preprint arXiv:2008.11110* (2020).

[26] Ian Fette and Alexey Melnikov. *The WebSocket Protocol*. 2011.

[27] Roy T. Fielding, Mark Nottingham, and Julian Reschke. *RFC 9110: HTTP Semantics*. 2022.

[28] James D. Foley. *Computer Graphics: Principles and Practice*. Vol. 12110. Addison-Wesley Professional, 1996.

[29] World Economic Forum. *Accelerating digital inclusion for 1 billion people by 2025*. 2024.

[30] Eric R. Fossum. "CMOS Image Sensors: Electronic Camera-On-A-Chip". In: *IEEE Transactions on Electron Devices* 44.10 (1997), pp. 1689–1698.

[31] Lovell Fuller. *Performance - sharp*. https://github.com/libvips/libvips/wiki/speed-and-memory-use. 2024.

[32] Thierry Gervais and Gaëlle Morel. *The Making of Visual News: A History of Photography in the Press*. Routledge, 2020.

[33] Samuel Ginzburg and Michael J Freedman. "Serverless Isn't Server-Less: Measuring and Exploiting Resource Variability on Cloud FaaS Platforms". In: *Proceedings of the 2020 Sixth International Workshop on Serverless Computing*. 2020, pp. 43–48.

[34] Robert L. Grossman. "The Case for Cloud Computing". In: *IT professional* 11.2 (2009), pp. 23–27.

[35] GSMA. *The Mobile Economy*. 2023.

[36] Dianyuan Han. "Comparison of Commonly Used Image Interpolation Methods". In: *Conference of the 2nd International Conference on Computer Science and Electronics Engineering (ICCSEE 2013)*. Atlantis Press. 2013, pp. 1556–1559.

[37] Jingning Han, Bohan Li, Debargha Mukherjee, Ching-Han Chiang, Adrian Grange, Cheng Chen, Hui Su, Sarah Parker, Sai Deng, Urvang Joshi, et al. "A technical overview of AV1". In: *Proceedings of the IEEE* 109.9 (2021), pp. 1435–1462.

[38] Joseph M. Hellerstein, Jose Faleiro, Joseph E. Gonzalez, Johann Schleier-Smith, Vikram Sreekanti, Alexey Tumanov, and Chenggang Wu. "Serverless Computing: One Step Forward, Two Steps Back". In: *arXiv preprint arXiv:1812.03651* (2018).

[39] David Hockney. *Secret Knowledge: Rediscovering the Lost Techniques of the Old Masters*. Thames & Hudson London, 2001.

[40] Roy Hoffman. *Data Compression in Digital Systems*. Springer Science & Business Media, 2012.

[41] Gerald C. Holst and Terrence S. Lomheim. *CMOS/CCD Sensors*. JCD publishing, 2007.

[42] Hugh Honour and John Fleming. *The Visual Arts: A History*. Pearson Education, 2010.

[43] Alain Hore and Djemel Ziou. "Image quality metrics: PSNR vs. SSIM". In: *2010 20th international conference on pattern recognition*. IEEE. 2010, pp. 2366–2369.

[44] Ping A. Hsieh and Ja-Ling Wu. "A Review of the Asymmetric Numeral System and Its Applications to Digital Images". In: *Entropy* 24.3 (2022), p. 375.

[45] David A. Huffman. "A Method for the Construction of Minimum-Redundancy Codes". In: *Proceedings of the IRE* 40.9 (1952), pp. 1098–1101.

[46] Kevin Hughes. “Entering the World-Wide Web: A guide to cyberspace”. In: *ACM SIGWEB Newsletter* 3.1 (1994), pp. 4–8.
[47] Robert William Gainer Hunt. *The Reproduction of Colour*. John Wiley & Sons, 2005.
[48] Eric Jonas, Johann Schleier-Smith, Vikram Sreekanti, Chia-Che Tsai, Anurag Khandelwal, Qifan Pu, Vaishaal Shankar, Joao Carreira, Karl Krauth, Neeraja Yadwadkar, et al. “Cloud Programming Simplified: A Berkeley View on Serverless Computing”. In: *arXiv preprint arXiv:1902.03383* (2019).
[49] Stefan Judis and Eric Portis. “Media”. In: *2004 Web Almanac* I.5 (2024).
[50] Michael Kavis. “Cloud Service Models”. In: *Architecting The Cloud*. John Wiley & Sons, Ltd, 2014. Chap. 2, pp. 13–22.
[51] M. Scott Kingsley. *Cloud Technologies and Services: Theoretical Concepts and Practical Applications*. Springer Nature, 2023.
[52] Michael F. Land and Dan-Eric Nilsson. *Animal Eyes*. Oxford University Press, 2012.
[53] Alex Libby. *Beginning SVG: A Practical Introduction to SVG using Real-World Examples*. Apress, 2018.
[54] Margaret Livingstone and David Hubel. “Segregation of Form, Color, Movement, and Depth: Anatomy, Physiology, and Perception”. In: *Science* 240.4853 (1988), pp. 740–749.
[55] Wes Lloyd, Shruti Ramesh, Swetha Chinthalapati, Lan Ly, and Shrideep Pallickara. “Serverless Computing: An Investigation of Factors Influencing Microservice Performance”. In: *2018 IEEE International Conference on Cloud Engineering (IC2E)*. IEEE. 2018, pp. 159–169.
[56] Pierre Magnan. “Detection of visible photons in CCD and CMOS: A comparative view”. In: *Nuclear Instruments and Methods in Physics Research Section A: Accelerators, Spectrometers, Detectors and Associated Equipment* 504.1 (2003), pp. 199–212.
[57] Guillaume C. Marty. *Optimising SVG images*. 2015.
[58] Lydia Meesters and Jean-Bernard Martens. “A single-ended blockiness measure for JPEG-coded images”. In: *Signal Processing* 82.3 (2002), pp. 369–387.
[59] Peter Mell and Tim Grance. *The NIST Definition of Cloud Computing*. 2011.
[60] Michael E. Mortenson. *Mathematics for Computer Graphics Applications*. Industrial Press Inc., 1999.
[61] James D. Murray and William VanRyper. *Encyclopedia of Graphics File Formats*. O’Reilly & Associates, Inc., 1996.
[62] Jakob Nielsen. “Response Times: The Three Important Limits”. In: *Usability Engineering* (1993).
[63] Gerard O’Regan. *The Innovation in Computing Companion*. Springer, 2018.
[64] Tim O’Reilly. *The Open Source Paradigm Shift*. 2003.
[65] John K. Ousterhout. “Scripting: Higher-Level Programming for the 21st Century”. In: *Computer* 31.3 (1998), pp. 23–30.
[66] J.D. van Ouwerkerk. “Image super-resolution survey”. In: *Image and Vision Computing* 24.10 (2006), pp. 1039–1052.
[67] Richard C. Pasco. *Source Coding Algorithms for Fast Data Compression*. Stanford University, 1976.
[68] Dana Petcu. “Portability and Interoperability between Clouds: Challenges and Case Study”. In: *Towards a Service-Based Internet: 4th European Conference*. Springer. 2011, pp. 62–74.

[69] Thomas Porter and Tom Duff. "Compositing Digital Images". In: *Proceedings of the 11th annual conference on Computer graphics and interactive techniques*. 1984, pp. 253–259.
[70] Hussachai Puripunpinyo and Mansur H. Samadzadeh. "Effect of Optimizing Java Deployment Artifacts on AWS Lambda". In: *2017 IEEE Conference on Computer Communications Workshops (INFOCOM WKSHPS)*. IEEE. 2017, pp. 438–443.
[71] Felix Richter. *Smartphones Cause Photography Boom*. 2017.
[72] Jorma Rissanen. "Modeling By Shortest Data Description". In: *Automatica* 14.5 (1978), pp. 465–471.
[73] Greg Roelofs. *PNG: The Definitive Guide*. O'Reilly & Associates, Inc., 1999.
[74] Alex Russell. *The Performance Inequality Gap*. 2024.
[75] David Salomon. *Data Compression: The Complete Reference*. Springer, 1998.
[76] Trever Schirmer, Nils Japke, Sofia Greten, Tobias Pfandzelter, and David Bermbach. "The Night Shift: Understanding Performance Variability of Cloud Serverless Platforms". In: *Proceedings of the 1st Workshop on Serverless Systems, Applications and Methodologies*. 2023, pp. 27–33.
[77] Mohammad Shahrad, Jonathan Balkind, and David Wentzlaff. "Architectural Implications of Function-as-a-Service Computing". In: *Proceedings of the 52nd annual IEEE/ACM international symposium on microarchitecture*. 2019, pp. 1063–1075.
[78] Claude E. Shannon. "A Mathematical Theory of Communication". In: *The Bell System Technical Journal* 27.3 (1948), pp. 379–423.
[79] Athanassios Skodras, Charilaos Christopoulos, and Touradj Ebrahimi. "The JPEG 2000 Still Image Compression Standard". In: *IEEE Signal Processing Magazine* 18.5 (2001), pp. 36–58.
[80] Dave Smart and Jamie Indigo. "Page Weight". In: *2004 Web Almanac* IV.16 (2024).
[81] Jon Sneyers. *JPEG XL's Modular Mode Explained*. 2024.
[82] Jon Sneyers, Elad Ben Baruch, and Yaron Vaxman. "CID22: Large-Scale Subjective Quality Assessment for High Fidelity Image Compression". In: *IEEE MultiMedia* (2023). DOI: 10.36227/techrxiv.22659061. Submitted.
[83] International Organization for Standardization. "Information technology – Cloud computing – Part 1: Vocabulary". Standard. 2023.
[84] Akamai Technologies. *Consumer Web Performance Expectations Survey*. 2014.
[85] Albert J.P. Theuwissen. *Solid-State Imaging with Charge-Coupled Devices*. Springer, 1995.
[86] Martin Thomson and Cory Benfield. *RFC 9113: HTTP/2*. 2022.
[87] Dmitrii Ustiugov, Theodor Amariucai, and Boris Grot. "Analyzing Tail Latency in Serverless Clouds with STeLLAR". In: *2021 IEEE International Symposium on Workload Characterization (IISWC)*. IEEE. 2021, pp. 51–62.
[88] Edward Wachtel. "The First Picture Show: Cinematic Aspects of Cave Art". In: *Leonardo* 26.2 (1993), pp. 135–140.
[89] Gregory K. Wallace. "The JPEG Still Picture Compression Standard". In: *Communications of the ACM* 34.4 (1991), pp. 30–44.
[90] Stefan Walraven, Eddy Truyen, and Wouter Joosen. "Comparing PaaS offerings in light of SaaS development: A comparison of PaaS platforms based on a practical case study". In: *Computing* 96 (2014), pp. 669–724.
[91] Liang Wang, Mengyuan Li, Yinqian Zhang, Thomas Ristenpart, and Michael Swift. "Peeking Behind the Curtains of Serverless Platforms". In: *2018 USENIX Annual Technical Conference (USENIX ATC 18)*. 2018, pp. 133–146.

[92] Yao Wang and Debargha Mukherjee. “The Discrete Cosine Transform and Its Impact on Visual Compression: Fifty Years From Its Invention [Perspectives]”. In: *IEEE Signal Processing Magazine* 40.6 (2023), pp. 14–17.

[93] Terry A. Welch. “A Technique for High-Performance Data Compression”. In: *Computer* 17.06 (1984), pp. 8–19.

[94] Ron White and Timothy E. Downs. *How Digital Photography Works*. Que Corp., 2005.

[95] Piotr Witkowski. *Practical FaaS Performance Variability Implications in the Serverless Image Optimization Scenario - Code Repository*. 2025.

[96] Qinzhe Wu and Lizy K. John. “Performance of Java in Function-as-a-Service Computing”. In: *2022 IEEE/ACM 15th International Conference on Utility and Cloud Computing (UCC)*. IEEE. 2022, pp. 261–266.

[97] Editors of Zdnet. *More digital than film cameras sold*. 2003.

[98] J. Zern, P. Massimino, and J. Alakuijala. “RFC 9649 WebP Image Format”. In: *Animation* 2 (2024), pp. 1–2.

[99] Jacob Ziv and Abraham Lempel. “A Universal Algorithm for Sequential Data Compression”. In: *IEEE Transactions on Information Theory* 23.3 (1977), pp. 337–343.

[100] Jacob Ziv and Abraham Lempel. “Compression of Individual Sequences via Variable-Rate Coding”. In: *IEEE Transactions on Information Theory* 24.5 (1978), pp. 530–536.